Classification of Financial News with Natural Language Processing: Machine and Deep Learning Methods

Sudenaz Ozel[1], Mert Ugur[2], and Gulay Cicek[3]
[1],[2],[3]Department of Software Engineering, Faculty of Engineering and Architecture
Istanbul Beykent University, Sarıyer, Istanbul, Türkiye
[1]sudenazozel0704@gmail.com, [2]ugurmert323@gmail.com
[3]gulaycicek@beykent.edu.tr

***Abstract*—In recent years, with the advancement of digital technologies, financial markets have become increasingly integrated into daily life and have started to respond rapidly to global events. However, the manual processing and interpretation of the large volume of financial news published daily pose significant challenges for investors. This study utilizes Machine Learning and Deep Learning techniques to classify financial news as positive, negative, or neutral. The dataset analyzed in this study, "Sentiment Analysis for Financial News," was obtained from the Kaggle platform and is commonly used for sentiment analysis of financial news. The dataset comprises news headlines with categorical sentiment labels (neutral, positive, negative). The dataset contains a total of 4,845 rows and two columns, including 2,878 neutral, 1,363 positive, and 604 negative samples. For preprocessing, text cleaning was performed, the TF-IDF method was used for feature extraction, and PCA and Variance Thresholding methods were employed for feature selection. Classification models were built using Naive Bayes, Support Vector Machines, Logistic Regression, Random Forest, K-Nearest Neighbors, Artificial Neural Networks, Long Short-Term Memory, Hierarchical Attention Networks, Convolutional Neural Networks, and Transformer algorithms. A unique modeling approach was developed by enhancing the ANN deep learning model. Performance evaluation metrics included accuracy, precision, specificity, F1-score, sensitivity, and AUC. K-Fold cross-validation was used. The results demonstrated that the proposed model outperformed other models in classifying news. Notably, the accuracy before PCA reached as high as 91.46This study introduced an original model alongside well-known machine learning and deep learning models for news classification. It highlighted that an original model could be effective for news classification and that preprocessing, feature selection and extraction techniques, and class imbalance mitigation methods enhance classification performance. Additionally, the use of multiple feature-processing and dimensionality-reduction approaches represents an innovative approach not found in previous studies. The study demonstrated that an advanced deep learning algorithm could be effective in news classification. Future studies plan to test the model on larger datasets and evaluate its applicability in non-financial sectors, such as real estate, energy, technology, and agriculture. Integrating the model into real-time analysis systems could support investors and industry professionals in real-time decision-making processes. Furthermore, integrating data from diverse sources such as social media, news websites, and industry reports could improve model accuracy. Testing its performance in regional markets within economic and cultural contexts is crucial for broadening its scope.**

***Keywords*—Financial Markets, Investor Decisions, Machine Learning, Deep Learning, News Classification, Accurate Information Access, Artificial Neural Networks**

I. Introduction

Financial markets play a critical role in the functioning of the global economy [18]. The process of globalization gained momentum with the collapse of the Bretton Woods system in the early 1970s [21]. The transition from fixed exchange rates to floating exchange rates created opportunities for profit through currency trading and paved the way for the emergence of high-volume foreign exchange markets [9].

With the advancement of technology, international capital flows have accelerated, and national markets have become increasingly integrated [34]. These markets support capital accumulation, contributing to corporate growth, government investments, and economic development [55]. They also offer investors opportunities to trade in assets such as stocks, bonds, currencies, and commodities [12]. Another function of financial markets is to maintain economic equilibrium, control costs, and reduce risks, thereby creating a sustainable investment environment.

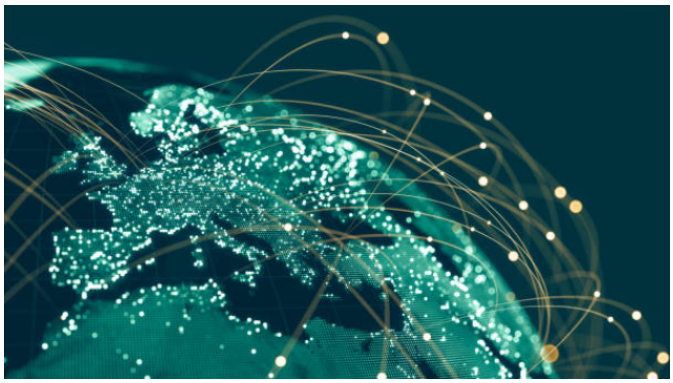


Fig. 1: Globalization in Markets (Figure [51])

With the continuous advancement of digital technologies, financial markets have become more integrated into daily life and have started to be influenced by global economic news more rapidly and directly [24]. The increasing use of smartphones, tablets, and other personal technological devices has caused global events to impact financial markets within extremely short periods [22].

This rapid and widespread flow of information brought about by digitalization has made financial movements more complex, creating both greater risks and opportunities for individual and institutional investors [22]. In such a dynamic environment, accurately analyzing news and predicting its effects on financial markets play a critical role in the success of investment processes. The strong relationship between news flow and markets directly influences the speed and accuracy of investment decisions. Delivering these analyses to investors quickly and reliably is vital for developing successful investment strategies. Misinterpreted or delayed information can lead to significant losses for investors, while accurate and timely analysis forms the foundation of profitable strategies

[23].

An investor is an individual or institution that allocates capital to different financial asset classes with the aim of generating profit [8]. By engaging in financial markets, investors assume risks and aim to benefit from the returns. Investors shape their decisions based on market movements, economic data, and financial news flow. These decisions typically vary depending on the level of risk they can take, the investment duration, and their goals [8].

Investors are generally categorized into two main groups: individual and institutional investors. Individual investors are those who invest their personal savings and usually have a more limited portfolio. Their risk tolerance depends on their personal financial situation. Institutional investors, on the other hand, are large-scale organizations such as banks, major corporations, investment funds, and pension funds [28]. They manage larger amounts of capital and typically follow long-term strategies.

Investors are the cornerstone of capital markets. They increase liquidity, support economic growth, and assist companies in raising resources. While the rise in individual investors allows for a broader distribution of capital, the strategic actions of institutional investors contribute to market stability and depth [11]. Although a large portion of the global population does not participate in financial markets as investors, the number of active individuals and institutions is steadily increasing (Figure 2).

In developed countries, the proportion of individual investors registered in stock markets can reach up to 50%, whereas in developing countries, this rate generally remains in the range of 5–20%. Institutional investors, however, account for a significant portion of total trading volume, shaping the fundamental dynamics of the markets [11]. Moreover, with the widespread adoption of technology and digitalization, the participation rate of individual investors is rapidly growing. The rise of online investment platforms and low-commission transactions has made access to financial markets easier for individuals worldwide.

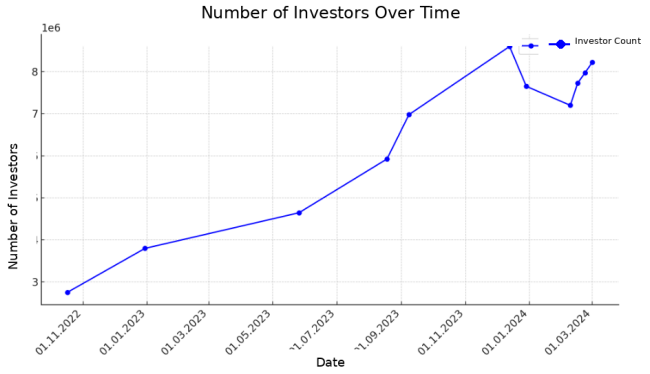


Fig. 2: Line Graph of the Number of Investors in Borsa Istanbul Over the Years (Figure [31])

The value of information is extremely high for individual investors who are active in financial markets or planning to invest [65]. For investors who lack access to information at both macro and micro levels or who are unable to interpret the information they access, such processes pose significant risks and cost factors [75].

As of 2024, the world population has reached approximately 8 billion, with around 22.8 million individuals participating in financial markets as investors. It is highly challenging for such a large number of investors to analyze financial news accurately and adapt it to their transactions. Every day, hundreds of thousands of news articles are published worldwide, and investors continue their activities without even seeing most of them [17]. Even if investors aim to read financial news, manually examining such a vast amount of information can result in significant time loss [17]. This hinders their ability to maximize profits and minimize losses. Moreover, manual review processes are inherently subjective and limited in reliability [45]. Consequently, the automatic classification of news facilitates quicker responses to market movements, enabling investors to react more efficiently and accurately.

This study employs classification methods based on Natural Language Processing (NLP) techniques to enable investors to classify financial news as positive, negative, or neutral [58]. NLP is a field of artificial intelligence that enables computers to understand and analyze human language [58]. It processes text or speech data using software and models designed to comprehend the complexity of human language [71]. By modeling the structures, semantic relationships, and context of language, NLP aims to produce accurate and meaningful results in natural language.

NLP techniques offer various methods for analyzing different aspects of language. These techniques include text classification, sentiment analysis, named entity recognition (NER), semantic analysis, language modeling, and machine translation [14]. Text classification allows the categorization of news as positive or negative [3]. Sentiment analysis identifies whether social media posts are positive or negative [62]. Language modeling ensures the coherent and meaningful generation of text.

NLP is widely utilized across various sectors. Common applications include search engines, voice assistants (e.g., Siri, Alexa), customer services, automatic text summarization, and machine translation. Additionally, it is effectively employed in functional areas such as medical text analysis in the healthcare sector, sentiment analysis in the financial sector, and automatic document analysis in the legal field [59].

The contributions of this study to the literature can be summarized as follows:

- Comprehensive Data Preprocessing Processes: Data preprocessing steps have been meticulously addressed, with detailed optimization of text cleaning, feature extraction, and dimensionality reduction processes. This has ensured more efficient operation of the classification models.
- Enhanced Feature Extraction and Dimensionality Reduction Techniques: Feature extraction and dimensionality reduction steps have been applied more effectively, optimizing both model performance and processing time.
- Algorithm Diversity and Performance Comparison: The study utilizes both traditional machine learning algorithms and deep learning models, conducting comparative analyses to reveal the strengths and weaknesses of each algorithm in detail.
- Configuration of Classification Algorithms: The study evaluates model configurations and regularization choices to improve performance while controlling overfitting.

The proposed future work aims to enhance model efficiency, accuracy, and the correct classification of more complex texts by leveraging NLP techniques alongside machine learning, deep learning, and data analytics methods [67] [72]. The machine learning methods employed include Naive Bayes (NB), Support Vector Machines (SVM), Logistic Regression (LR), Random Forest (RF), and K-Nearest Neighbors (KNN). For deep learning architectures, Artificial Neural Networks (ANN), Long Short-Term Memory (LSTM) [27], Hierarchical Attention Networks (HAN),

Convolutional Neural Networks (CNN), and Transformer models have been utilized [6] [26] [4].

Machine learning algorithms have been utilized to transform texts into meaningful features and for classification processes, while deep learning algorithms are preferred for their ability to learn complex relationships in large datasets. Data analytics plays a critical role in preprocessing the dataset and addressing class imbalance through methods like TF-IDF, PCA, and SMOTE.

The structure of the paper is planned as follows:

Section II. Literature Review: This section will review and compare existing studies in the field.

Section III. Methodology: It will explain how the system will be implemented and detail the classification algorithms used.

Section IV. Experimental Results: Common metrics in the literature, such as accuracy, precision, sensitivity, F1 score, ROC curve, and confusion matrix, will be used for performance evaluation [66].

Section V. Discussion: This section will discuss the limitations of the study and goals for future development.

Section VI. Conclusion: A general conclusion statement about the paper will be provided.

II. Literature Review

In the literature, many studies involving machine and deep learning methods for classifying news as positive and negative have been examined. Additionally, studies related to comment classification, fake news detection, and text classification have been reviewed to aid in the context of this study. The reviewed studies are summarized in Tables I, II, and III under the headings of authors, datasets, sample sizes, methods, results, limitations, and future contributions. These reviewed studies have helped identify shortcomings in classification studies, and the algorithm has been shaped based on these deficiencies.

Emrah et al. (2021) conducted a study on news classification using a dataset of 2,248 samples comprising eight categories collected from an online Turkish news website. They employed Multinomial Naive Bayes (MNB) and Random Forest (RF) algorithms. The highest success was achieved with the RF algorithm, yielding an accuracy of 99.6%. The dataset used in this study is limited in size and diversity, and only two classification algorithms were employed. In future studies, enriching the dataset and including news texts in different languages could enhance classification performance. The use of different algorithms and hybrid models in such studies could allow for a comparative analysis of results, enabling more efficient classification [10].

Çiğdem et al. (2019) conducted a study using 3,600 samples across six categories from the Kaggle-TCC-3600 dataset for news classification. Initially, Word2Vec was applied for data preprocessing, enriching the dataset. Two Convolutional Neural Network (CNN) models were used, with the first CNN model achieving a higher accuracy of 93.3%. This study is significant in demonstrating that CNNs perform better in text classification applications compared to traditional machine learning and data mining algorithms. However, the dataset size is limited, and only two models of a single algorithm were utilized. Future studies could use larger datasets, incorporate different algorithms, or increase the number of CNN models to achieve a model with higher accuracy [1].

Özer et al. (2021) performed a classification study using 12,000 news samples across six categories from six different news websites. The raw data were preprocessed to remove unnecessary words and spaces, and vector models were created using TF-IDF, Word2Vec, and FastText methods. These vector models were classified using Support Vector Machines (SVM), Naive Bayes (NB), Logistic Regression (LR), Random Forest (RF), and Artificial Neural Networks (ANN). The highest accuracy rate of 95.75% was achieved using the FastText method with SVM. The dataset size in this study is limited, and although ANN was used, it is insufficient in terms of deep learning methods. Future studies could focus on expanding the dataset, employing hybrid models in preprocessing, and comparing classification results using different deep learning models to enhance performance [20].

Hülya et al. (2023) classified 4,200 news texts across seven categories using the Kaggle-Turkish Headlines and BBC News Classification datasets. Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and GloVe-derived GloVe-CNN and GloVe-LSTM methods were employed. The accuracy of the LSTM model improved from 81% to 91% when the GloVe embedding method was used. However, no such improvement was observed in the CNN model. The highest accuracy rate of 97% was achieved with the CNN model. The study highlights the insufficiency of the dataset size. Future studies should be conducted on larger datasets and include comparative analyses of models trained with different hyperparameters [44].

Halil et al. (2024) developed a model to classify 4,027 news texts across nine categories using the Turkish *newsTRT2023* dataset. For vector representation of the data, a BERT-based *BertTurk* model was used, enabling high-dimensional and meaningful representations of the texts and enhancing natural language understanding. Graph Neural Networks (GNN) were then used for classification, achieving a high accuracy of 97.93%. This study provides an innovative contribution to Turkish text classification and relational analysis fields. However, class imbalance in the dataset may negatively affect prediction performance for certain classes. Future studies should aim to create more balanced datasets to improve classification performance [52].

Mehmet et al. (2023) demonstrated that fake news detection can be performed quickly and successfully by using 250 tweets out of a total of 5,500 samples obtained from the Twitter social network. They applied the TF-IDF method for feature extraction and weighting. Decision Trees and Naive Bayes algorithms were used as classification models, and the results were evaluated using a confusion matrix. The highest success was achieved with the NB algorithm, yielding an accuracy of 88.4%. The number of samples used in this study is small, and the number of algorithms employed is insufficient. Future studies aim to include news from different sources, expand the dataset, and utilize various machine learning methods [45].

Esra et al. (2021) created a classification model using 3,000 user reviews obtained from Amazon, IMDB, and Yelp websites. In this study, K-Nearest Neighbors (KNN), Naive Bayes (NB), Support Vector Machines (SVM), Decision Trees (DT), and Random Forest (RF) algorithms were used. Initially, each algorithm was run separately on the datasets, and an increase in model performance was observed when all datasets were combined. The best result was achieved with the SVM algorithm, yielding an accuracy of 85%. This study did not employ deep learning methods, and the dataset size is limited. Future studies aim to conduct more comprehensive sentiment analysis by increasing dataset diversity and size and employing deep learning-based classification algorithms [19].

Önder et al. (2022) conducted a sentiment classification study using 15,082 samples obtained from YouTube user comments. In this classification performed with the Naive Bayes algorithm, the data were categorized into three sentiment classes: positive (5,167),

Authors (Year)	Dataset	Number of Samples	Methods	Results	Limitations	Future Contributions
Emrah Aydemir, Murat Işık, and Türker Tuncer (2021) [10]	Turkish online news website	2,248 samples	SVM, RO	Accuracy: 99.86% F-Score: 99.9% (highest result with RO)	Dataset is limited. Few classification algorithms were used.	The study should be conducted with a larger dataset and additional algorithms for better results.
Çiğdem Acı and Adem Çırak (2019) [1]	Kaggle - TCC-3600 dataset	3,600 samples	SVM	Accuracy: 93.3% F-Score: 90%	Only one classification algorithm was used. Dataset is limited.	Multiple algorithms should be used, and the dataset should be expanded to increase the number of samples.
Özer Çelik and Burak Can Koç (2021) [20]	6 different news websites	12,000 samples	SVM, NB, LR, RO, ANN	Accuracy: 95.75% (highest result with FastText and SVM)	No comparison with other classification studies. Dataset is limited. Metrics such as F-Score were not calculated.	Comparisons with other studies in the field should be made, and the number of samples should be increased. Metrics calculations should also be included.
Hülya Hark, Meral Karakurt, Cengiz Hark, and Ali Karcı (2023) [39]	Kaggle - Turkish Headlines dataset and BBC News Classification dataset [42]	4,200 samples (600 samples per category: economy, life, politics, technology, entertainment, health, sports)	TF-IDF, ESA	Accuracy: 97% Precision: 97% Recall: 99% F-Score: 98% (highest result with ESA)	Dataset is limited.	Larger datasets should be used, and different models should be developed.
Halil İbrahim Okur, Kadir Tohma, and Ahmet Sertbaş (2024) [52]	Turkish newsTRT2023 dataset	4,027 samples (World: 1,567, Turkey: 1,442, Economy: 438, Others: 2713)	TF-IDF, SVM	Accuracy: 97.93% Precision: 95.08% Recall: 92.22% F-Score: 93.59%	Dataset is imbalanced and limited.	Dataset imbalance should be addressed, and the number of samples should be increased.
Mehmet Karakuş and Fatma Yiğit Açıkgöz (2023) [45]	Twitter	5,500 samples	NB, SVM	Accuracy: 88.4% Precision: 89.3% Recall: 87.2% F-Score: 88.3% (highest result with NB)	The number of samples and categories are insufficient. Algorithms used are limited. No comparison with other studies.	The number of samples should be increased, and comparisons should be made using additional algorithms.
Esra Çelik, Deniz Dal, and Tolga Aydın (2021) [19]	Amazon, IMDb, and Yelp	3,000 samples (500 positive and 500 negative samples from each website)	TF-IDF, NB, SVM, ANN, RO	Accuracy: 85% Recall: 85% Precision: 85.1% F-Score: 85% (highest result with SVM)	Deep learning was not used. Dataset diversity and size are limited.	The dataset should be expanded, and classification using deep learning algorithms should be conducted.
Önder Şahinaslan, Hüseyin Dalyan, and Ender Şahinaslan (2022) [62]	YouTube comments dataset	15,082 samples	NB	Accuracy: 65.56%	Few classification algorithms were used. Metrics such as Precision and F-Score were not calculated.	The number of algorithms used should be increased, and additional metrics should be calculated.
Cengiz Hark (2022) [38]	COVID-19 dataset	10,700 samples	TF-IDF, ESA, TSA, RO	Accuracy: 91% F-Score: 92% (highest success achieved with ESA)	Dataset is limited.	The number of samples should be increased.
Feyza Altunbey Özbay and Bilal Alataş (2020) [53]	ISOT dataset	44,898 samples (Real: 21,417, Fake: 23,481)	NB, JRip, J48, RO, SVM, ANN, Gradient Boosting	Accuracy: 97.5% Precision: 97.6% Recall: 97.6% (highest result with RO)	Deep learning was not used. No comparison with other advanced algorithms.	Different algorithms should be tested, and comparisons should be provided.

TABLE I: Literature Review Results

Article Authors (Year)	Dataset	Number of Samples	Methods	Results	Limitations	Future Contributions
Doğancan Kınık and Aysun Güran (2021) [48]	1150News dataset, Hürriyet6c-1k dataset, Milliyet-9c-1k dataset, AA News dataset	36,150 Samples	NB, KNN, LR, RF, DT, SVM	Accuracy:94% (Best results with LR, RF, SVM)	No comparison was made with other advanced algorithms.	Comparisons with different algorithms should be made, and the number of samples should be increased.
Süleyman Gökhan Taşkın, Ecir Uğur Küçüksille, and Kamil Topal (2021) [69]	Twitter	76,738 Samples	KNN, SVM, RF	F1-Score: 86% (Best result with RF)	The dataset is limited.	Different word representation methods should be used, and the number of samples should be increased.
Yunus Santur (2020) [64]	Hepsiburada	243,000 Samples (98% Positive, 2% Negative)	ESA, LSTM, GTB	Accuracy:95.9% (Best result with GTB)	The dataset is imbalanced.	The number of algorithms used should be increased, and a balanced dataset should be created.
Gözde Nergiz and Erdinç Avaroğlu (2021) [50]	Twitter, Instagram, Youtube	180,000 Samples	LSTM	Accuracy:93.15%	Machine learning methods were not used. No comparison was made with other algorithms.	Different algorithms should be tested and compared.
Ali Dayan and Atınç Yılmaz (2022) [25]	Kaggle-Audio Cats and Dogs dataset	277 Samples (Dog sounds: 113, Cat sounds: 164)	LSTM, ESA	Accuracy:91% (Best result with LSTM)	The dataset is limited.	The number of classification algorithms and samples should be increased.
Mustafa Özkan and Görkem Kar (2022) [56]	National Thesis Center Database	1,560 Samples	BERT	Accuracy:97% Precision:100% Recall:98% F1-Score:99%	Machine learning was not used. The dataset is limited.	Different algorithms should be tested, and the number of samples should be increased.
Ahmet Albayrak and Adham Jolosı Jolosı Zada (2023) [76]	An e-commerce platform	2,237 Samples	RF, AdaBoost	Accuracy:87.9%	Deep learning was not used.	Different algorithms should be used to increase diversity, and the number of samples should be increased.
Yasin Görmez, Halil Arslan, and Bilal Atak (2024) [36]	Winvoker and Beyazperde	500,306 Samples (Winvoker Positive: 262,166, Negative: 56,561, Neutral: 170,917, Beyazperde Positive: 5,331, Negative: 5,331)	ESA	Accuracy:94.16% Precision:91.75% Recall:89.58% F1-Score:90.58% (Best result with Transformer and ESA hybrid model)	Machine learning was not used. No comparison was made with other advanced algorithms.	Different algorithms should be tested, and comparisons should be made.
Nurcan Yücel and Özge Cömert (2023) [74]	Hepsiburada, n11, and Trendyol	2,000 Samples (Positive: 1,000, Negative: 1,000)	LSTM	Accuracy:91.5% Specificity:86.92% Recall:96.77% F1-Score:91.37%	Machine learning was not used. No comparison was made with other advanced algorithms.	Different algorithms should be tested, and the number of samples should be increased.
Zeynep Hilal Kilimci (2020) [47]	Twitter	54,052 Samples (Negative: 38,529, Positive: 12,047)	ESA, TSA, LSTM	Accuracy:82.27% Specificity:78.92% Sensitivity:79.96% (Best result with English dataset and STCK)	The dataset is imbalanced. The number of algorithms is insufficient.	A balanced dataset should be used, and different algorithms should be tested.

TABLE II: Literature Review Results

Article Authors (Year)	Dataset	Number of Samples	Methods	Results	Limitations	Future Contributions
Mehmet Umut Salur and İlhan Aydın (2022) [63]	Twitter	14,777 Samples (Positive: 6,887, Negative: 7,890)	LSTM	Accuracy:75.71% Sensitivity:65.37% F1-Score: 71.20% Kappa: 51.48% ROC-AUC: 74.93% (Highest success with BiLSTM)	Machine learning was not used.	Different algorithms should be tested, and the number of samples should be increased.
Abdülkadir Tepecik and Engin Demir (2022) [70]	Common Voice	20,760 Samples	NB, SVM, RF, LR, KNN	Accuracy:69.60% F1-Score:71% Sensitivity:71% (Highest results with SVM)	No comparison was made with other advanced algorithms.	The number and diversity of samples should be increased.
Serpil Aslan (2022) [7]	Twitter	176,345 Samples	SVM, RF, LR	Accuracy:94% Precision:96% Recall:95% F1-Score: 96%	No information was provided about the dataset's positivity or negativity.	The use of hybrid models should be increased.
Mesut Toğaçar, Kamil Abdullah Eşidir, and Burhan Ergen (2022) [71]	Kaggle-REAL or FAKE News Dataset	6,335 Samples (Real: 3,171, Fake: 3,164)	LSTM	Accuracy:91.48% Specificity:92.88% Sensitivity:92.76% Recall:92.88% F1-Score:91.55%	Machine learning was not used.	Different algorithms should be tested, and the number and diversity of samples should be increased.
Zeynep Orman and Mehmet Can Yılmaz (2021) [73]	Twitter	485,000 Samples	LSTM	Accuracy:99%	Deep learning was not used. No comparison was made with other advanced algorithms.	Different algorithms should be tested, and data preprocessing should be more detailed.
Anıl Utku, Serim Gençaslan, and M. Ali Akcayol (2020) [35]	UCF-101 Dataset	13,320 Samples	CNN, VGG-19	Accuracy:83% Precision:85% Recall:82% F1-Score: 83% (Highest results with VGG-19)	No comparison was made with other advanced algorithms.	Hybrid models should be used, and the number of samples should be increased.
Murat İnce (2021) [41]	MS-COCO Dataset	300,000 Samples	TSA, CNN	Accuracy:81.95% Precision:81.12% Recall:82% F1-Score: 81.33%	The diversity of the dataset is limited.	Examples in different languages should be included in the study.
Ebru Somuncu and Nesrin Aydın Atasoy (2022) [68]	Synth90k Dataset	90,000	Hybrid LSTM, ESA, TSA, ETSA	Accuracy:96% (Highest result with ETSA)	The dataset is limited.	Improvements on algorithms should be used to increase accuracy.
Enver Yazğılı and Muhammet Baykara (2022) [72]	Kaggle	3,000 Samples (Positive: 1,503, Negative: 1,497)	RF, C-4.5, SVM, NB, LR, ANN, KNN, CNN	Accuracy:88.35% Precision:88.93% Sensitivity:87.75% F1-Score:88.33% (Best results with LR)	The dataset is insufficient. Deep learning was not used.	The number of samples should be increased, and deep learning algorithms should be used.

TABLE III: Literature Review Results

negative (4,644), and neutral (5,271). An analysis conducted with WEKA resulted in an accuracy rate of A noteworthy aspect of this study is that the comments were translated into different languages for classification, contributing to multilingual sentiment analysis. Future studies are recommended to use more advanced algorithms such as Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Long Short-Term Memory (LSTM), Artificial Neural Networks (ANN), and BERT. Additionally, tools like the Yandex API could be utilized for translation into different languages [62].

Cengiz (2022) conducted a study to classify fake news using a dataset of 10,700 samples from the COVID-19 fake news dataset. The study employed Multi-Layer Perceptron (MLP), Long Short-Term Memory (LSTM), Recurrent Neural Networks (RNN), and Convolutional Neural Networks (CNN) algorithms. In the initial phase, the GloVe word embedding technique was applied for fake news detection, followed by classification algorithms. This enabled a comprehensive evaluation of performances for a challenging and contemporary task like fake news detection. The study aimed to address a significant gap in the field by comparing the effectiveness of different word embedding methods and neural network-based classifiers. Future work is expected to achieve more realistic and consistent classification results by increasing the number of samples [38].

Feyza et al. (2020) developed a classification model for fake news detection using 44,898 samples from the ISOT dataset. The dataset was divided into four experimental groups, and 10 different artificial intelligence algorithms were used. These algorithms included Naive Bayes (NB), Jrip, J48, Random Forest (RF), Stochastic Gradient Descent (SGD), Locally Weighted Learning (LWL), Naive Bayes with Decision Tree (NBTree), Bootstrap Aggregation (Bagging), and Classification via Regression (CVR). The highest accuracy rate of 97.5% was achieved with the Random Forest algorithm. This study is one of the first to examine the collective performance of supervised artificial intelligence algorithms on a real dataset. Future studies are anticipated to achieve better results by incorporating deep learning algorithms. Additionally, more efficient and effective systems for fake news detection are planned to be developed [53].

Doğancan et al. (2021) created a classification model using a dataset of 36,150 samples from the 1150Haber, Hürriyet6c-1k, and Milliyet-9c-1k datasets. Initially, TF-IDF was used for word weighting, followed by word embeddings using Word2Vec and Doc2Vec. The classification was conducted using Naive Bayes (NB), K-Nearest Neighbors (KNN), Logistic Regression (LR), Support Vector Machines (SVM), Decision Trees (DT), and Multi-Layer Perceptrons (MLP). The best results were obtained with TF-IDF weighting and LR, SVM, and MLP algorithms. This study contributed to the literature by demonstrating that word weighting and embedding before classification improve success rates [48].

Süleyman et al. (2021) developed a classification model for fake news detection using 76,738 samples collected from Twitter. TF-IDF, Word2Vec, and Doc2Vec methods were employed for word representation, and classification was performed using K-Nearest Neighbors (KNN), Support Vector Machines (SVM), and Random Forest (RF) algorithms. This study's significant contribution is enabling the detection of fake news within a very short time. Moreover, analyzing the followers and followed accounts of users on social media platforms could help evaluate whether the accounts are fake. Future studies could improve algorithm performance and F1 scores by incorporating follower data into automated detection systems [69].

Serpil (2022) developed a sentiment analysis classification model using 176,345 tweets about the COVID-19 vaccine collected from Twitter. A BiGRU-CNN hybrid model was created using GloVe and TF-IDF word embeddings, and the data were classified with this model. Classification algorithms such as Convolutional Neural Networks (CNN), Support Vector Machines (SVM), Random Forest (RF), and Logistic Regression (LR) were used. The best results were achieved with the BiGRU-CNN hybrid model, obtaining an accuracy of 94%. This study demonstrates that the use of hybrid models improves accuracy. However, error analysis of the model was not conducted. Future studies plan to develop a deep learning-based system to detect fake accounts and misinformation on social media [7].

Mesut et al. (2022) created a model for classifying 6,335 news articles as fake or real using the Kaggle-REAL or FAKE News Dataset, Long Short-Term Memory (LSTM), natural language processing, and GloVe word embeddings. An accuracy of 91.48% was achieved. This study highlights that using hybrid models can increase success rates. However, more deep learning and machine learning algorithms could have been tested. Future studies aim to analyze the detection of fake news and misinformation on datasets created from Turkish news websites using different approaches [71].

Zeynep et al. (2021) developed a classification model for sentiment analysis using 485,000 Turkish tweets collected during the COVID-19 period. In the preprocessing phase, unnecessary tweets and words were removed, and the Long Short-Term Memory (LSTM) model was used to achieve a high classification accuracy of 99%. However, a sufficiently comprehensive sentiment lexicon was not used, and tweets not containing words from the lexicon were excluded, resulting in data loss. Future studies aim to enhance model performance by using a more comprehensive sentiment lexicon and labeling with a larger dataset [73].

Anıl et al. (2023) created a model for event classification using 13,320 videos from the UCF-101 dataset. Convolutional Neural Networks (CNNs) initially did not achieve sufficient success, so transfer learning was applied with the VGG-19 model to improve performance. This new model achieved an accuracy of 83%. Future studies could involve testing additional algorithms [35].

Murat (2021) developed a model combining Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Natural Language Processing (NLP) methods to extract metadata from 300,000 samples in the MS-COCO dataset. Objects in the images were encoded with CNNs, and textual equivalents of these encoded features were generated with RNNs. This model achieved an accuracy of 81.95%. The study presents an innovative system that combines deep learning and natural language processing techniques to automate the process of generating metadata from images. Future studies aim to enable automatic content generation using these automatically generated and classified metadata [41].

Ebru et al. (2022) developed a model for character recognition on 90,000 text images from the Synth90k dataset. The algorithms used included Bidirectional Long Short-Term Memory (BiLSTM), Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and the Convolutional Recurrent Neural Network (CRNN) formed by combining CNNs and RNNs. The best results were achieved with CRNN, initially yielding an accuracy of 94%, which was later improved to 96% using the Levenshtein Distance (LD). These results were compared with similar studies in the literature, achieving high success with 96% accuracy. Since the Synth90k dataset mainly contains English text, this may limit the dataset and

accuracy. Future studies aim to develop indoor navigation systems using the CRNN architecture for mobile devices [68].

Enver et al. (2022) used a manually created dataset of 3,000 Turkish sentences obtained from Kaggle to detect cyberbullying using natural language processing techniques. Algorithms such as Random Forest (RF), C4.5, Support Vector Machines (SVM), Naive Bayes (NB), Logistic Regression (LR), Artificial Neural Networks (ANN), K-Nearest Neighbors (KNN), and K-Means Clustering (KMC) were applied. The highest accuracy of 88.35% was achieved with the Logistic Regression (LR) algorithm. The novelty of the dataset and the testing of many algorithms make this study a significant contribution to the literature. The challenges posed by the morphological structure of the Turkish language in NLP applications were highlighted. Future studies plan to use deep learning algorithms and enable real-time detection of cyberbullying on social media platforms [72].

III. METHOD

A. Dataset

1) Data Source: In this study, the "Sentiment Analysis for Financial News" dataset, obtained from the Kaggle platform and used for sentiment analysis of financial news, was utilized [77]. The dataset consists of publicly available financial news and is used for classification problems.

2) Data Type: The dataset consists of two columns. The first column is a categorical data type containing sentiment labels (neutral, positive, negative) for the news headlines. The second column contains the news headlines as text (string) data.

3) Data Size: The dataset contains a total of 4,845 rows and 2 columns. The distribution of sentiment labels is as follows: 2,878 neutral (59.4%), 1,363 positive (28.1%), and 604 negative (12.5%). This indicates that the dataset has an imbalanced structure.

B. Data Preprocessing

Data preprocessing is the process of transforming raw data into a form suitable for machine learning and data analytics. This process involves removing irrelevant details, filling gaps, scaling, and preparing the data for modeling. The objective is to create more accurate, effective, and efficient models. The importance of data preprocessing includes improving data quality, enhancing model performance, increasing computational efficiency, and ensuring generalizability.

In our study, the following steps were performed:

Text cleaning, including converting text to lowercase, removing punctuation marks, and cleaning numbers. Feature extraction using TF-IDF to convert text into numerical data. Removing low-variance features using the Variance Threshold method. Encoding sentiment classes into numerical formats, labeling them as 0, 1, and 2. These steps ensured that the data was prepared for modeling and provided a significant step toward achieving accurate and reliable results in data analytics, machine learning, and other data-driven tasks.

C. Scaling

Scaling involves transforming the numerical values in the dataset into a specific range (e.g., between 0 and 1) or a specific distribution. Ensuring that data is appropriately scaled is particularly important for achieving better performance in certain machine learning algorithms.

The purpose of scaling includes:

Improving Model Performance: Ensuring balanced and efficient learning when all features are on the same scale. Avoiding Incorrect Distance Calculations: Adjusting data on different scales to prevent inaccuracies in distance-based algorithms such as K-Nearest Neighbors (KNN) and Support Vector Machines (SVM). Among scaling methods, Min-Max Scaling (Normalization) was used to transform data into a specific range, typically [0,1].

Formula:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \tag{1}$$

and Z-Score Scaling (Standardization), which transforms the data to have a mean of 0 and a standard deviation of 1, has been examined.

Formula:

$$X' = \frac{X - \mu}{\sigma} \tag{2}$$

Here, μ represents the mean, and σ represents the mean, and

However, when the news headlines in our dataset are transformed into numerical vectors using TF-IDF, these vectors are typically within the same range (e.g., 0 to 1). Since TF-IDF vectors are already normalized, additional scaling is generally not required.

D. Feature Selection

Feature selection is the process of identifying the features in a dataset that most significantly affect the model's performance and are most important for analysis. This process aims to create a more efficient and accurate model by eliminating unnecessary and redundant features in the dataset.

*1) **TF-IDF Method**:* The TF-IDF (Term Frequency-Inverse Document Frequency) method was used for feature extraction from news headlines. This method identifies words in the news headlines as features and calculates the importance of each word in a headline by considering its frequency (TF) and rarity across other headlines (IDF). As a result of this process, the 5,000 most frequently used words were selected as features, and each news headline was represented by a TF-IDF vector consisting of 5,000 features. The resulting TF-IDF feature matrix is of size (4845, 5000), where the rows represent news headlines and the columns represent word-based features.

*2) **Variance Thresholding (VT)**:* To select important features and reduce model complexity, Feature Importance and Variance Thresholding methods were examined. Feature Importance ranks features (variables) based on their impact on the target variable and selects the most significant features according to this ranking. This method is often used in models like decision trees and random forests to identify the impact of features on the target variable. In Variance Thresholding, the variance of features is measured, and those with low variance are removed from the dataset. A feature is selected only if it exceeds a certain variance threshold; low-variance features are typically considered to carry little information for the model. Using the Variance Thresholding method, we calculated the variance of the features and removed those with low variance. As a result of this process, low-variance features were removed from the TF-IDF feature matrix.

*3) **Principal Component Analysis (PCA)**:* Principal Component Analysis (PCA) is a technique that aims to capture the most important information in a high-dimensional dataset by transforming it into a lower-dimensional form. In this study, PCA was used in Natural Language Processing (NLP) to reduce the dimensions of text data and simplify word vectors.

The feature matrix obtained after feature selection was reduced to 100 principal components using PCA. Two different experiments were conducted to compare model performance with and without PCA. It was observed that the accuracy before applying PCA was higher.

E. Dataset Splitting and K-Fold cross-validation

Dataset splitting is a fundamental step in machine learning that involves dividing the dataset into two or more parts for use during the training and testing phases. Typically, the dataset is divided into a training set and a test set. The training set is used for the model to learn and derive relationships, while the test set is applied to evaluate the model's performance on unseen data.

K-Fold cross-validation is an advanced method used to evaluate model performance more reliably. In this approach, the dataset is divided into

K equal parts (folds). Over K iterations, each fold is used as the test set once, while the remaining K-1 folds are used as the training set. At the end of this process, each data point is included once in the test set and K-1 times in the training set. The performance results from K iterations are averaged to measure the model's generalization ability more reliably.

The main difference between dataset splitting and K-Fold cross-validation lies in the data splitting method. Simple dataset splitting divides the data into training and test sets once, whereas K-Fold cross-validation splits the dataset multiple times in different ways. By using the entire dataset for both training and testing, K-Fold provides a more comprehensive evaluation. This method is particularly advantageous in cases where the dataset is small or there are class imbalances.

Data splitting methods other than K-Fold cross-validation, such as the Train-Test Split method, which divides the dataset into training and test sets in fixed proportions for model evaluation; the LOOCV method, which uses each data point as the test set in turn and the remainder for training; and the Shuffle Split method, which randomly splits the dataset into training and test sets at a specified ratio, have been examined. However, other methods were not used because a broader evaluation was desired with K-Fold cross-validation.

The K-Fold cross-validation method was chosen to evaluate the model's performance. In the study, K = 5 was determined, and the dataset was divided into 5 equal parts in each iteration. In each iteration, 4 parts were used for training, and 1 part was used for testing to calculate accuracy rates. The results of the evaluations with and without PCA are summarized as follows: Without PCA, Average accuracy: 76.57%, Standard deviation: 0.61%; With PCA, Average accuracy: 71.76%, Standard deviation: 0.63%. The results show that higher accuracy was achieved without PCA.

F. Model Selection

Machine Learning

Machine learning is a mathematical model or method that enables a computer to learn from data and use this learning to perform a specific task [43]. Machine learning algorithms are used to assign input data to a specific class or label [46]. Important aspects of machine learning include generalization, autonomous operation, improvement over time, automatic learning, and enhancement capabilities. The machine learning algorithms used in this study are Naive Bayes, Support Vector Machine, Logistic Regression, Random Forest, and K-Nearest Neighbors.

*1) **Naive Bayes (NB)**:* The Naive Bayes Classifier is a simple yet effective classification algorithm based on Bayes' Theorem [6] and is widely used in machine learning applications [10]. It is particularly preferred in natural language processing (NLP) applications, such as text classification, language detection, and document similarity tasks. This model is called "naive" because it assumes that each feature is independent of the others, which is not true for real-world data. However, this simplified assumption yields quite good results in practice. Naive Bayes works in conjunction with some fundamental techniques in the NLP field. These include the "bag of words" technique, which vectorizes the data by treating words as unordered features [61]; the "TF-IDF (Term Frequency – Inverse Document Frequency)" technique, which measures the importance of words [20]; and the "feature selection" technique, which improves model performance by selecting only specific words as features in some cases [40]. Naive Bayes is a probabilistic model, and class predictions are directly based on probability calculations. Therefore, it does not use any linear or non-linear activation functions. Advantages of Naive Bayes compared to other algorithms include its speed and simplicity, low memory consumption, high success rate, and efficiency. Disadvantages include the assumption that features are independent, which is not always accurate, and its limitations in working with categorical data while generally performing better with numerical data.

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)} \tag{3}$$

- $P(C|X)$: The conditional probability of class C given the data X.
- $P(X|C)$: The probability of the data X under class C.
- $P(C)$: The prior probability of class C.
- $P(X)$: The overall probability of the data X.

*2) **Support Vector Machines (SVM)**:* Support Vector Machines (SVM) is an effective supervised learning algorithm used in classification analyses [26]. SVM aims to find the optimal linear separator that divides the data into different classes. The number of layers is zero. It transforms each data point into a high-dimensional space to better separate the boundaries between the data. The main components of SVM are the "hyperplane," which separates data into two classes; "support vectors," which are the data points closest to the hyperplane and used for classification; and the "kernel function," which allows for separation in higher dimensions [49]. SVM is used in natural language processing (NLP) tasks such as text mining, sentiment analysis, and text classification. The advantages of SVM compared to other algorithms include better performance on high-dimensional data, generalization capability, and good results on non-linear data. However, its disadvantages include complexity in tuning, slow performance on large datasets, and preference for majority classes in cases of class imbalance [57].

$$\mathbf{w} \cdot \mathbf{x} + b = 0 \tag{4}$$

Here:

- $\mathbf{w}$: Normal vector of the hyperplane (weight vector),
- $\mathbf{x}$: Data point,
- b: Bias term.

*3) **Logistic Regression (LR)**:* Logistic Regression is a simple and linear model commonly used for classification problems. Despite its name "regression," it primarily performs classification. LR is widely used for binary classification problems [15]. For example, in our context of news classification, news articles are

either positive or negative. The linear and simple nature of LR makes the model easy to understand and enhances interpretability. It operates faster than more complex models, making it advantageous for quick results in large datasets. Logistic Regression is typically a single-layer model. The basic structure of the model consists of a linear function followed by a sigmoid activation function to produce the output. Therefore, the number of layers in Logistic Regression is 1.

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b) \tag{5}$$

Here:

- $\hat{y}$: The probability predicted by the model (a value between 0 and 1).
- $\sigma(z) = \frac{1}{1+e^{-z}}$: The **sigmoid** function, which converts the linear output of the model into a probability between 0 and 1.
- $\mathbf{w}$: Weight vector (parameters learned by the model).
- $\mathbf{x}$: Feature vector (independent variables).
- b: Bias term (a parameter learned during training).

*4) **Random Forest (RF)**:* Random Forest is a robust, flexible, and resilient machine learning algorithm. As an ensemble of decision trees, it provides an effective solution for both classification and regression problems [13]. It works with high accuracy on large datasets and minimizes overfitting. However, its interpretability may be weak, and computational costs can be high. Combining multiple decision trees often yields much better results than a single decision tree, improving the accuracy of the RF algorithm. It minimizes overfitting by using Bootstrap Aggregation and creates diverse trees [60]. In Random Forest, the "number of layers" is not explicitly defined. Instead, the number of decision trees can be thought of as equivalent to the number of layers. Thus, combining a large number of decision trees produces a stronger and more reliable model.

For Classification:

$$\hat{y} = \text{mode}\left(f_1(x), f_2(x), \ldots, f_M(x)\right) \tag{6}$$

Here:

- $\hat{y}$: Final prediction (determined by majority voting).
- $f_i(x)$: Prediction of the i-th decision tree.
- M: Total number of decision trees (ensemble size).

*5) **K-Nearest Neighbor (KNN)**:* KNN is a supervised learning algorithm. It predicts the class or value of a data point based on the classes or values of its K nearest neighbors [2]. For a given test data point, it assigns a class or numerical value by examining the nearest neighbors in the training data. KNN adapts quickly to datasets because it does not require a complex model-building or learning process and is particularly effective for small datasets. It is applicable to both classification and regression problems, offering flexibility depending on the data type and problem [29]. KNN is a simple algorithm, making it easy to understand and implement. The number of layers in KNN is 1 because the model makes predictions by calculating distances between the test sample and other samples in the training dataset.

Euclidean distance formula:

$$d(x_1, x_2) = \sqrt{\sum_{i=1}^{n} (x_{1i} - x_{2i})^2} \tag{7}$$

Based on the computed distances, neighbors are selected:

- **For Classification:** Majority voting is performed among the classes of the neighbors:

$$\hat{y} = \text{mode}(y_1, y_2, \ldots, y_K)$$

  Here, $y_1, y_2, \ldots, y_K$ are the class labels of the K neighbors.
- **For Regression:** The average of the neighbors' labels is taken as the prediction:

$$\hat{y} = \frac{1}{K} \sum_{i=1}^{K} y_i$$

  Here, y_i are the class values of the neighbors.

Deep Learning

Deep learning is a subfield of artificial intelligence and machine learning that refers to the process of learning from data, particularly using multi-layered structures (typically artificial neural networks). It enables the development of models capable of making high-accuracy predictions on complex data structures and large datasets through advancements in neural networks [54].

Deep learning aims to learn hidden patterns in data by using multi-layered neural networks that function similarly to the neurons in the human brain [54]. This method has achieved significant success in fields such as image processing, speech recognition, natural language processing (NLP), and autonomous vehicles.

Key aspects of deep learning include providing high accuracy and performance, automatic feature extraction, effective handling of large datasets, and the ability to learn and improve automatically. The deep learning algorithms used in this study include Artificial Neural Networks, Long Short-Term Memory, Hierarchical Attention Networks, Convolutional Neural Networks, and Transformers.

*6) **Artificial Neural Networks (ANN)**:* Artificial Neural Networks mimic the functioning of neurons in the human brain and are used in data analysis, classification, regression, and similar tasks in machine learning and deep learning fields [5]. ANNs are powerful in learning from large datasets and modeling complex relationships. By learning patterns in the training data, ANNs can generalize, make accurate predictions on new data, and be applied to a wide range of problems. They are effective in various domains, such as image processing, speech recognition, and text analysis. ANNs can contain multiple hidden layers. Activation functions in ANNs process neuron outputs in a non-linear manner, enabling the learning of complex data. One of the most commonly used activation functions is the sigmoid [5].

Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{8}$$

- Limits the output between 0 and 1, making it particularly useful for binary classification problems.

*7) **Long Short-Term Memory (LSTM)**:* Long Short-Term Memory is a type of recurrent neural network architecture that excels with sequential data and specializes in analyzing and extracting meaning from groups of words in texts [4]. It is specifically designed to overcome challenges in learning long-term dependencies in sequential data. Compared to standard Recurrent Neural Networks (RNNs), LSTM is more advanced [30]. To address the "vanishing gradient problem" encountered in other networks, it uses a special structure called cell states and gates [32]. These gates control which information is retained, stored, or updated, enabling it to remember information over long sequences. LSTM works with some techniques in NLP, including "text classification" for categorizing sentiments as positive or negative, "language

modeling and text generation" for calculating the likelihood of a text sequence and generating text, "machine translation" for automatic translation between languages, and "sentiment analysis" for determining the sentiment of a text or document. Advantages of LSTM compared to other algorithms include its ability to learn long-term dependencies, its solution to the vanishing gradient problem, and its ability to recall past data in long sequences for decision-making. Disadvantages include high computational cost, susceptibility to overfitting with small datasets [37], and limited utility for short-term relationships. The number of LSTM layers can be one (single-layer) or multiple, depending on the model depth and complexity of the data.

Forget Gate The forget gate determines how much of the previous cell state will be retained. It is calculated using the following formula:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{9}$$

Here:

- f_t: Output of the forget gate.
- W_f: Weight matrix for the forget gate.
- b_f: Bias for the forget gate.
- h_{t-1}: Output (hidden state) from the previous time step.
- x_t: Current input.

*8) **Hierarchical Attention Networks (HAN)**:* Hierarchical Attention Networks (HAN) is a deep learning model specifically designed for the classification of long texts, processing text at word, sentence, and paragraph levels. The word-level attention mechanism selects important words, while the sentence-level attention mechanism focuses on critical sentences in constructing the meaning of the text. This hierarchical structure enables superior performance in long texts by learning both local and global contexts. The use of attention layers allows the model to focus on important sections of the text while reducing the impact of irrelevant information. Unlike transformer-based models like BERT, HAN processes texts in a hierarchical structure, making it advantageous for long texts. However, it has high computational costs and is heavily dependent on the dataset. It is frequently used in tasks such as text classification, document analysis, and summarization.

Word-Level Attention Mechanism

The HAN model first learns the representation of words within sentences. The attention mechanism in this process ensures a focus on important words. The formulas are:

$$u_{it} = \tanh(W_w h_{it} + b_w)$$

$$\alpha_{it} = \frac{\exp(u_{it}^\top u_w)}{\sum_t \exp(u_{it}^\top u_w)}$$

$$s_i = \sum_t \alpha_{it} h_{it}$$

- h_{it}: Hidden state of the t-th word in the i-th sentence.
- W_w, b_w, u_w: Learnable parameters of the word-level attention mechanism.
- α_{it}: Importance weight of the t-th word.
- s_i: Representation vector of the i-th sentence.

Sentence-Level Attention Mechanism

Sentence representation vectors (from word-level s_i) are processed through the sentence-level attention mechanism to select important sentences for understanding the text. The formulas are:

$$u_i = \tanh(W_s s_i + b_s)$$

$$\alpha_i = \frac{\exp(u_i^\top u_s)}{\sum_i \exp(u_i^\top u_s)}$$

$$v = \sum_i \alpha_i s_i$$

- s_i: Representation vector of the i-th sentence.
- W_s, b_s, u_s: Learnable parameters of the sentence-level attention mechanism.
- α_i: Importance weight of the i-th sentence.
- v: Final representation vector of the text, used for classification or other NLP tasks.

*9) **Convolutional Neural Network (CNN)**:* Convolutional Neural Networks are a deep learning algorithm widely used in fields such as image processing and video analysis [16]. CNNs use convolutional layers to learn the spatial structure and local features of data. These layers are optimized to discover important features in the input. CNNs work with high accuracy on large datasets and are generally used in tasks such as image classification, object detection, face recognition, and image segmentation, consisting of three layers [33]. These layers are the convolutional layer, pooling layer, and fully connected layer [33]. CNNs automatically learn important features from images without requiring manual feature extraction. This eliminates the need for feature engineering and provides translational invariance by learning similar features in different regions of the data.

The convolutional operation is expressed as follows:

$$S(x, y) = (I * F)(x, y) = \sum_i \sum_j I(x + i, y + j) \cdot F(i, j) \tag{14}$$

Here:

- I: Input image.
- F: Filter (kernel).
- S: Output feature map.

*10) **Transformer**:* The Transformer algorithm, developed in 2017 by Vaswani et al., is a revolutionary deep learning model in the field of natural language processing (NLP). The Transformer overcomes the limitations of traditional recurrent neural networks and long short-term memory models when processing sequential data. Its most significant innovation is the use of the attention mechanism, which allows the model to learn the relationship of each word with all other words. The multi-layered structure of the Transformer enables much deeper and more abstract learning of data. Additionally, it is a flexible model that can be used for other types of sequential data such as audio and images.

Attention Mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{15}$$

Here:

- d_k: Represents the dimensionality of the Query and Key vectors.
- The softmax function normalizes the attention weights.

G. Model Originality

In addition to the known machine and deep learning algorithms reviewed in the literature, the ANN-based approach used in this study was enhanced with the following methods:

- Combining TF-IDF for text feature extraction with a two-step approach involving PCA and Variance Thresholding for dimensionality reduction. This combination is rarely used for sentiment analysis.
- Using a fixed hyperparameter configuration together with dropout regularization to improve performance and address overfitting and underfitting. The reported custom ANN configuration uses 10 epochs, a batch size of 32, a dropout rate of 0.3, and a learning rate of 0.001.
- Addressing class imbalance problems to prevent the model from favoring certain classes when there are significant differences between the dataset's class distributions. SMOTE was used to generate synthetic data points for minority classes.

The algorithm used in this study provides a unique approach to the sentiment analysis problem of financial news. The algorithm was chosen for its ability to perform classification with high accuracy while offering solutions tailored to the dataset's structure. Combining TF-IDF with dimensionality reduction techniques like PCA and Variance Thresholding represents a powerful yet underutilized method in the literature. This approach reduces computational cost while improving performance by transforming high-dimensional textual features into compact representations. Additionally, the selected hyperparameter configuration and dropout regularization were used to control overfitting and support generalization. SMOTE increased the representation of minority classes, enabling balanced learning from imbalanced datasets. This algorithm's importance lies in its ability to optimize accuracy and generalization in addressing complex problems like sentiment analysis. Its advantages include computational efficiency, class balance, and achieving high accuracy with tailored hyperparameters.

H. Model Training

The model training process was carried out using an artificial neural network (ANN). The process involved training the model separately with data before and after applying PCA. The training steps included the following:

1) Defining the ANN Model: The model was created using the 'create_ann' function, which defines the layers and hyperparameters of the model:

• Layers:

- First hidden layer: 128 neurons, ReLU activation function.
- Second hidden layer: 64 neurons, ReLU activation function.
- Output layer: Softmax activation function for classification.

• Regularization:

- A 30% dropout rate was applied after each hidden layer (dropout_rate=0.3) to prevent overfitting.

• Optimization:

- The Adam optimization algorithm (learning_rate=0.001) was used to dynamically adjust the learning rate for fast and stable optimization.

• Loss Function:

- The categorical_crossentropy loss function, suitable for multi-class classification, was used.

• Evaluation Metrics:

- Model performance was evaluated using the accuracy metric.

2) Hyperparameter Configuration: Candidate values were defined for experimental comparison, while the reported public experiment uses the fixed configuration listed below:

- Number of epochs: Determines how many times the entire dataset is seen by the model (epochs=[10, 20]).
- Batch size: Number of examples provided to the model in each iteration (batch_size=[16, 32, 64]).
- Learning rate: Controls the rate of weight updates (learning_rate=[0.001, 0.01]).

3) Training Process: The training process was conducted separately for data before and after applying PCA.

For data before PCA, the dataset was split into training and test sets using Stratified K-Fold cross-validation with 5 folds. The data was prepared in a format suitable for the model.

For data after PCA, the same process was repeated with dimensionality-reduced data. In each fold, the model was trained using the training data. Using the 'fit' method, the model was optimized for the specified epochs and batch sizes. Setting verbose=0 suppressed training outputs. Training was performed by optimizing the model weights based on the categorical_crossentropy loss function.

4) Prediction and Performance Evaluation: The trained model was applied to the test dataset to generate predictions. Performance was measured using metrics such as accuracy, precision, recall, F1-score, AUC, and specificity.

5) Comparison of Results Before and After PCA: Performance results were recorded separately for data before and after PCA. The average of all metrics was calculated to evaluate the model's performance.

In conclusion, model training was conducted using cross-validation both with and without PCA using the selected fixed hyperparameter configuration. The training process was evaluated comprehensively with metrics such as accuracy, precision, recall, AUC, and F1-score, analyzing the impact of PCA.

I. Evaluation Metrics

Several metrics used to evaluate the performance of the model include accuracy, sensitivity, specificity, F1-score, and AUC-ROC. These metrics are critical in measuring the success of the model.

Confusion Matrix: The confusion matrix is a table that shows the relationship between the predicted classes and the actual classes. It can be represented as follows:

	True Positive (TP)	True Negative (TN)
Predicted Positive (FP)	TP	FP
Predicted Negative (FN)	FN	TN

Here:

- **TP** (True Positive): Cases where the model correctly predicts the positive class.
- **TN** (True Negative): Cases where the model correctly predicts the negative class.
- **FP** (False Positive): Cases where the model incorrectly predicts the positive class (Type I error).
- **FN** (False Negative): Cases where the model incorrectly predicts the negative class (Type II error).

Performance Metrics

The following are some key metrics used to evaluate the model's performance:

Accuracy: Accuracy represents the proportion of correct predictions made by the model:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{16}$$

Sensitivity (Recall): Sensitivity measures the proportion of actual positives that are correctly identified:

$$\text{Sensitivity} = \frac{TP}{TP + FN} \tag{17}$$

Specificity: Specificity measures the proportion of actual negatives that are correctly identified:

$$\text{Specificity} = \frac{TN}{TN + FP} \tag{18}$$

F-Score: The F-Score is the harmonic mean of sensitivity and precision:

$$F\text{-Score} = 2 \times \frac{\text{Sensitivity} \times \text{Precision}}{\text{Sensitivity} + \text{Precision}} \tag{19}$$

Precision: Precision is the ratio of true positive predictions to all positive predictions:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{20}$$

AUC-ROC: AUC-ROC is a graph used to evaluate the classification success of the model. The ROC (Receiver Operating Characteristic) curve shows the relationship between the True Positive Rate (TPR) and the False Positive Rate (FPR). AUC (Area Under Curve) represents the area under this curve and measures the model's overall performance:

$$\text{AUC} = \int_0^1 TPR\, dFPR \tag{21}$$

IV. EXPERIMENTAL RESULTS

This study aims to develop an artificial intelligence solution for classifying financial news as positive, negative, or neutral. The dynamic structure of financial markets necessitates the rapid and accurate analysis of the impact of news on market movements. In this context, natural language processing (NLP) techniques and various machine learning and deep learning models were utilized to effectively classify financial news.

The dataset used in the study is the "Sentiment Analysis for Financial News" dataset, sourced from the Kaggle platform, consisting of financial news collected from publicly available sources for sentiment analysis. The dataset contains a total of 4,845 rows and 2 columns. The distribution of sentiment labels is as follows: 2,878 neutral, 1,363 positive, and 604 negative, indicating that the dataset is imbalanced. In our study, text preprocessing steps include converting text to lowercase, removing punctuation, and cleaning numbers. Texts were transformed into numerical data using feature extraction with TF-IDF.

The preprocessing steps applied to the dataset include converting text to lowercase, removing punctuation marks, and cleaning numbers. During the feature extraction and selection stages, methods such as TF-IDF, PCA, and Variance Thresholding were employed.

For the classification process, machine learning methods such as Naive Bayes, Support Vector Machines (SVM), Logistic Regression (LR), Random Forest (RF), and K-Nearest Neighbors (KNN) were utilized, in addition to deep learning algorithms such as Artificial Neural Networks (ANN), Long Short-Term Memory (LSTM), Hierarchical Attention Networks (HAN), Convolutional Neural Networks (CNN), and Transformer models. Our custom algorithm was developed based on the ANN deep learning algorithm. The custom algorithm incorporates detailed data preprocessing and advanced feature extraction with dimensionality reduction techniques.

This section evaluates and compares the performance of the applied methods using metrics such as accuracy, precision, recall, and F1 score. The results of the most successful models are examined in detail. Our study aims to present a model that provides accurate and rapid classification, offering a more reliable and efficient information analysis process for investors.

A. Model Performance and Evaluation

*1) **Performance Metrics**:* Performance metrics play a critical role in assessing the accuracy and effectiveness of a model. Accuracy represents the ratio of correctly predicted examples to all predictions, while sensitivity, or the true positive rate, measures the model's ability to correctly predict the positive class. Specificity represents the ability to correctly identify the negative class. Precision indicates how many of the positive predictions are truly positive, and the F1-score balances sensitivity and precision, summarizing the overall performance of the model. Finally, AUC evaluates the model's ability to distinguish between positive and negative classes at various threshold levels. When considered together, these metrics provide a comprehensive insight into the model's performance, highlighting its ability to make accurate and balanced predictions, which is particularly crucial for imbalanced datasets.

***Impact of PCA on Model Performance**:* Principal Component Analysis (PCA) is an effective method used to reduce the dimensionality of high-dimensional data, simplifying the analysis process. One of PCA's main advantages is its ability to uncover hidden relationships in the dataset while removing meaningless variations in the data structure. Additionally, PCA is often preferred in machine learning models for its ability to lower computational costs and help prevent overfitting. In this study, 100-component PCA was used for dimensionality reduction. However, the results showed that dimensionality reduction with PCA decreased model performance, indicating that PCA was not suitable for this dataset.

***Impact of Variance Thresholding on Model Performance**:* Variance thresholding is a method that removes low-variance features from the dataset, reducing input dimensions, preventing overfitting, and lowering computational costs. However, as shown in Table 4, applying variance thresholding resulted in a significant decrease in model performance. For instance, the original model's accuracy (81.23%), precision (81.20%), F1-score (81.17%), and AUC (93.31%) values were lower compared to scenarios with and without PCA. This indicates that low-variance features can sometimes carry useful information, and their removal may harm the model's generalization ability. While variance thresholding offers benefits in dimensionality reduction and computational efficiency, it should be applied cautiously in cases where high accuracy and precision are required. The results suggest that the impact of this method on performance should be carefully evaluated based on the selected threshold value.

*2) **Comparison**:* In this section, the results of machine learning and deep learning algorithms are compared in detail with the results of our hybrid model. Bar graphs (Figures 3 and 4) were used for comparison, and each value represented in the graph is explained.

The results obtained with the NB algorithm were as follows: 69.31% accuracy, 71.81% precision, 76.92% specificity, 63.63%

TABLE IV: Naive Bayes Classification Results

Naive Bayes (PCA Enabled)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	59.75	63.51	66.92	45.02	59.75	74.09
Fold2	59.44	35.33	66.66	44.32	59.44	75.96
Fold3	59.44	35.33	66.66	44.32	59.44	75.84
Fold4	59.55	63.46	66.83	44.66	59.54	75.74
Fold5	59.96	63.60	67.14	45.58	59.95	75.45
Average	**59.63**	**52.25**	**66.85**	**44.78**	**59.62**	**75.42**
Naive Bayes (PCA Disabled)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	69.25	71.87	77.12	63.84	69.24	82.88
Fold2	69.76	72.84	77.14	64.07	69.76	83.84
Fold3	67.91	69.06	75.73	61.31	67.90	83.68
Fold4	70.07	73.00	77.48	64.76	70.07	82.00
Fold5	69.56	72.26	77.13	64.18	69.55	81.43
Average	**69.31**	**71.81**	**76.92**	**63.63**	**69.30**	**82.77**
Naive Bayes (Variance Threshold Enabled)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	59.75	63.51	66.92	45.02	59.75	72.57
Fold2	59.44	35.33	66.67	44.32	59.44	74.29
Fold3	59.44	35.33	66.67	44.32	59.44	74.94
Fold4	59.44	63.42	66.75	44.43	59.44	73.76
Fold5	59.96	63.60	67.17	45.58	59.95	73.83
Average	**59.61**	**52.24**	**66.83**	**44.74**	**59.61**	**73.88**

TABLE V: Support Vector Machine Classification Results

Support Vector Machine (PCA ON)						
Fold	**Accuracy(%)**	**Precision(%)**	**Specificity(%)**	**F-Score(%)**	**Recall(%)**	**AUC(%)**
Fold1	71.41	72.03	38.10	67.95	71.41	81.37
Fold2	71.10	72.17	36.54	67.29	71.10	82.74
Fold3	69.76	70.72	36.56	65.67	69.76	81.39
Fold4	72.03	73.07	33.98	68.25	72.03	81.05
Fold5	70.38	71.20	36.27	66.43	70.38	81.40
Average	**70.94**	**71.84**	**36.29**	**67.12**	**70.94**	**81.59**
Support Vector Machine (PCA OFF)						
Fold	**Accuracy(%)**	**Precision(%)**	**Specificity(%)**	**F-Score(%)**	**Recall(%)**	**AUC(%)**
Fold1	77.40	77.22	58.18	76.47	77.40	87.84
Fold2	77.50	77.50	61.61	76.42	77.50	88.07
Fold3	76.68	76.93	61.47	75.46	76.68	88.46
Fold4	77.19	77.25	56.07	75.85	77.19	87.14
Fold5	76.99	76.98	59.09	75.80	76.99	88.01
Average	**77.15**	**77.18**	**59.28**	**76.00**	**77.15**	**87.91**
Support Vector Machine (Variance Threshold ON)						
Fold	**Accuracy(%)**	**Precision(%)**	**Specificity(%)**	**F-Score(%)**	**Recall(%)**	**AUC(%)**
Fold1	70.59	71.87	39.45	66.69	70.59	79.71
Fold2	70.38	71.83	34.31	65.96	70.38	80.40
Fold3	71.10	73.48	38.61	66.95	71.10	79.78
Fold4	70.38	71.39	31.00	65.84	70.38	79.31
Fold5	69.35	70.63	33.96	64.77	69.35	79.31
Average	**70.36**	**71.84**	**35.47**	**66.04**	**70.36**	**79.70**

TABLE VI: Logistic Regression Classification Results

Logistic Regression (PCA ON)						
Fold	**Accuracy(%)**	**Precision(%)**	**Specificity(%)**	**F-Score(%)**	**Recall(%)**	**AUC(%)**
Fold1	71.62	71.47	42.00	69.33	71.62	82.28
Fold2	72.45	72.69	41.58	69.53	72.45	83.18
Fold3	70.69	71.58	41.94	67.78	70.69	81.62
Fold4	72.34	72.29	35.00	69.46	72.34	82.74
Fold5	71.72	71.49	41.00	69.02	71.72	82.42
Average	**71.76**	**71.90**	**40.30**	**69.02**	**71.76**	**82.45**
Logistic Regression (PCA OFF)						
Fold	**Accuracy(%)**	**Precision(%)**	**Specificity(%)**	**F-Score(%)**	**Recall(%)**	**AUC(%)**
Fold1	76.78	77.21	51.82	75.28	76.78	87.51
Fold2	77.61	78.77	53.70	75.94	77.61	87.75
Fold3	76.57	78.27	49.52	74.65	76.57	87.81
Fold4	75.95	76.25	47.12	74.03	75.95	87.11
Fold5	75.95	76.61	50.00	74.19	75.95	87.73
Average	**76.57**	**77.42**	**50.43**	**74.82**	**76.57**	**87.58**
Logistic Regression (Variance Threshold ON)						
Fold	**Accuracy(%)**	**Precision(%)**	**Specificity(%)**	**F-Score(%)**	**Recall(%)**	**AUC(%)**
Fold1	71.52	71.75	41.90	68.93	71.52	80.60
Fold2	72.03	72.85	39.22	68.77	72.03	81.36
Fold3	71.31	72.45	39.18	68.01	71.31	80.30
Fold4	72.14	72.59	33.66	68.88	72.14	81.00
Fold5	71.00	70.90	36.54	68.05	71.00	80.33
Average	**71.60**	**72.11**	**38.10**	**68.53**	**71.60**	**80.72**

TABLE VII: Random Forest Classification Results

Random Forest (PCA ON)						
Fold	**Accuracy(%)**	**Precision(%)**	**Specificity(%)**	**F-Score(%)**	**Recall(%)**	**AUC(%)**
Fold1	72.14	72.89	42.72	69.14	72.14	76.95
Fold2	70.79	72.51	33.67	66.40	70.79	78.13
Fold3	69.97	70.67	37.08	65.90	69.97	77.30
Fold4	71.00	72.18	29.90	66.90	71.00	79.70
Fold5	69.87	71.10	32.69	65.56	69.87	79.81
Average	**70.75**	**71.87**	**35.21**	**66.78**	**70.75**	**78.38**
Random Forest (PCA OFF)						
Fold	**Accuracy(%)**	**Precision(%)**	**Specificity(%)**	**F-Score(%)**	**Recall(%)**	**AUC**
Fold1	75.44	76.69	48.15	73.08	75.44	86.20
Fold2	74.10	75.33	43.56	71.18	74.10	86.40
Fold3	75.34	77.70	47.52	72.60	75.34	85.46
Fold4	74.61	76.38	37.96	71.70	74.61	85.59
Fold5	73.27	75.48	41.67	69.97	73.27	86.17
Average	**74.55**	**76.32**	**43.77**	**71.71**	**74.55**	**85.97**
Random Forest (Variance Threshold ON)						
Fold	**Accuracy(%)**	**Precision(%)**	**Specificity(%)**	**F-Score(%)**	**Recall(%)**	**AUC(%)**
Fold1	72.03	73.03	38.68	68.88	72.03	76.86
Fold2	69.87	71.00	32.65	65.75	69.87	76.98
Fold3	71.21	72.69	41.30	67.47	71.21	76.77
Fold4	71.21	72.49	29.34	67.16	71.21	78.03
Fold5	71.00	72.91	33.00	67.14	71.00	78.05
Average	**71.06**	**72.43**	**34.99**	**67.28**	**71.06**	**77.34**

TABLE VIII: K-Nearest Neighbors Classification Results

K-Nearest Neighbors (PCA ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	68.42	66.71	43.00	65.70	68.42	72.36
Fold2	68.52	67.13	41.51	65.24	68.52	73.48
Fold3	69.35	67.95	46.94	66.03	69.35	73.26
Fold4	67.49	65.70	36.54	64.09	67.49	73.41
Fold5	67.29	65.34	42.72	64.18	67.29	73.24
Average	**68.21**	**66.57**	**42.14**	**65.05**	**68.21**	**73.15**
K-Nearest Neighbors (PCA OFF)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	70.28	69.20	59.05	69.05	70.28	80.10
Fold2	68.94	67.56	63.46	67.38	68.94	78.75
Fold3	72.24	71.26	69.07	70.93	72.24	80.86
Fold4	69.25	68.50	58.42	68.36	69.25	79.57
Fold5	65.74	64.43	62.89	64.38	65.74	77.22
Average	**69.29**	**68.19**	**62.58**	**68.02**	**69.29**	**79.30**
K-Nearest Neighbors (Variance Threshold ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	66.67	64.55	36.08	63.67	66.67	70.95
Fold2	68.42	67.55	39.25	65.08	68.42	71.88
Fold3	69.25	67.84	44.55	66.44	69.25	72.70
Fold4	67.39	65.76	31.68	64.11	67.39	71.67
Fold5	67.18	65.65	39.62	64.01	67.18	71.59
Average	**67.78**	**66.27**	**38.24**	**64.66**	**67.78**	**71.76**

TABLE IX: Artificial Neural Networks Classification Results

Artificial Neural Networks (PCA ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	74.41	73.88	61.11	73.35	74.41	84.63
Fold2	74.51	73.56	60.64	73.24	74.51	85.68
Fold3	73.17	72.61	58.59	71.57	73.17	83.87
Fold4	74.10	73.41	53.00	72.30	74.10	85.12
Fold5	74.82	74.19	64.42	73.95	74.82	84.89
Average	**74.20**	**73.53**	**59.55**	**72.88**	**74.20**	**84.84**
Artificial Neural Networks (PCA OFF)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	76.68	76.17	64.86	76.11	76.68	86.11
Fold2	74.41	74.50	65.09	74.23	74.41	84.60
Fold3	73.48	72.86	67.29	72.97	73.48	85.44
Fold4	74.82	74.33	70.48	74.44	74.82	86.01
Fold5	74.92	74.61	71.70	74.72	74.92	85.27
Average	**74.86**	**74.49**	**67.88**	**74.50**	**74.86**	**85.49**
Artificial Neural Networks (Variance Threshold ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	73.27	72.51	55.96	71.99	73.27	82.84
Fold2	73.37	72.63	55.34	71.81	73.37	83.44
Fold3	73.07	73.13	52.00	70.80	73.07	82.97
Fold4	74.20	74.05	45.54	71.92	74.20	83.73
Fold5	73.79	73.50	53.70	72.16	73.79	82.96
Average	**73.54**	**73.16**	**52.51**	**71.73**	**73.54**	**83.19**

TABLE X: Long Short-Term Memory Classification Results

LSTM (PCA ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	57.89	46.45	0.88	46.94	57.89	54.86
Fold2	55.11	45.81	0.93	48.01	55.11	55.91
Fold3	52.32	45.22	5.50	47.02	52.32	54.88
Fold4	51.81	47.87	4.48	49.17	51.81	54.89
Fold5	53.56	41.91	0.00	45.72	53.56	54.20
Average	**54.14**	**45.45**	**2.36**	**47.37**	**54.14**	**54.95**
LSTM (PCA OFF)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	73.99	73.48	57.14	73.15	73.99	82.73
Fold2	74.41	73.89	69.81	73.99	74.41	82.46
Fold3	74.30	73.78	70.79	73.44	74.30	82.10
Fold4	75.95	75.35	69.44	75.38	75.95	84.24
Fold5	74.92	74.73	71.11	74.41	74.92	83.02
Average	**74.72**	**74.25**	**67.66**	**74.07**	**74.72**	**82.91**
LSTM (Variance Threshold ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	54.08	44.78	3.51	46.58	54.08	53.27
Fold2	57.38	48.21	3.39	47.41	57.38	55.25
Fold3	50.98	42.07	2.93	44.40	50.98	53.66
Fold4	49.33	43.49	1.00	45.92	49.33	53.72
Fold5	56.14	45.18	6.14	46.50	56.14	54.04
Average	**53.58**	**44.75**	**3.40**	**46.29**	**53.58**	**53.99**

TABLE XI: Convolutional Neural Network Classification Results

CNN (PCA ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**Recall (%)**	**F-Score (%)**	**AUC (%)**
Fold1	68.32	67.02	79.37	63.07	68.32	78.53
Fold2	68.63	67.63	76.81	64.72	68.63	78.99
Fold3	67.80	69.81	76.78	62.66	67.80	79.89
Fold4	70.28	70.84	79.02	66.69	70.28	79.38
Fold5	66.46	63.79	75.20	61.86	66.46	77.71
Average	**68.30**	**67.82**	**77.44**	**63.80**	**68.30**	**78.90**
CNN (PCA OFF)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**Recall (%)**	**F-Score (%)**	**AUC (%)**
Fold1	75.95	75.69	86.43	75.79	75.95	87.63
Fold2	75.03	75.18	84.82	75.10	75.03	86.16
Fold3	75.13	74.73	84.81	74.84	75.13	87.70
Fold4	72.96	72.94	83.51	72.92	72.96	85.30
Fold5	74.41	74.15	84.19	74.22	74.41	86.26
Average	**74.70**	**74.54**	**84.75**	**74.57**	**74.70**	**86.61**
Convolutional Neural Network (Variance Threshold ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**Recall (%)**	**F-Score (%)**	**AUC (%)**
Fold1	68.52	69.32	77.27	66.53	68.52	77.36
Fold2	67.29	67.39	77.48	62.33	67.29	78.60
Fold3	68.32	68.67	74.21	64.32	68.32	77.51
Fold4	69.56	70.97	79.89	65.05	69.56	77.46
Fold5	67.18	66.60	77.27	62.41	67.18	76.84
Average	**68.17**	**68.59**	**77.22**	**64.13**	**68.17**	**77.55**

TABLE XII: Hierarchical Attention Networks Classification Results

Hierarchical Attention Networks (PCA ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	59.44	35.33	0.00	44.00	59.44	50.31
Fold2	59.44	35.33	0.00	44.32	59.44	50.05
Fold3	59.44	35.33	0.00	44.32	59.44	50.00
Fold4	59.34	35.21	0.00	44.20	59.39	50.00
Fold5	59.33	35.21	0.00	44.19	59.33	49.06
Average	**59.40**	**35.29**	**0.00**	**44.27**	**59.40**	**49.88**
Hierarchical Attention Networks (PCA OFF)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	59.44	35.33	0.00	44.32	59.44	49.31
Fold2	59.44	35.33	0.00	44.32	59.44	49.99
Fold3	59.44	35.33	0.00	44.32	59.44	49.97
Fold4	59.33	35.21	0.00	44.20	59.33	49.93
Fold5	59.33	35.21	0.00	44.20	59.33	50.07
Average	**59.40**	**35.29**	**0.00**	**44.27**	**59.40**	**49.86**
Hierarchical Attention Networks (Variance Threshold ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	59.44	35.33	0.00	44.32	59.44	50.07
Fold2	59.44	35.33	0.00	44.32	59.44	50.00
Fold3	59.44	35.33	0.00	44.32	59.44	49.74
Fold4	59.33	35.21	0.00	44.20	59.33	49.99
Fold5	59.33	35.21	0.00	44.20	59.33	50.32
Average	**59.40**	**35.29**	**0.00**	**44.27**	**59.40**	**50.02**

TABLE XIII: Transformer Classification Results

Transformer (PCA ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F1-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	59.44	35.33	66.67	44.32	59.44	49.65
Fold2	59.44	35.33	66.67	44.32	59.44	49.95
Fold3	59.44	35.33	66.67	44.32	59.44	50.12
Fold4	59.34	35.21	66.67	44.20	59.34	50.06
Fold5	59.34	35.21	66.67	44.20	59.34	49.55
Average	**59.40**	**35.29**	**66.67**	**44.27**	**59.40**	**49.87**
Transformer (PCA OFF)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F1-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	59.44	35.33	66.67	44.32	59.40	49.82
Fold2	59.44	35.33	66.67	44.32	59.44	50.00
Fold3	59.44	35.33	66.67	44.32	59.44	50.38
Fold4	59.34	35.21	66.67	44.20	59.34	49.89
Fold5	59.34	35.21	66.67	44.20	59.34	50.07
Average	**59.40**	**35.29**	**66.67**	**44.27**	**59.40**	**50.03**
Transformer (Variance Threshold ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F1-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	59.44	35.33	66.67	44.32	59.44	50.02
Fold2	59.44	35.33	66.67	44.32	59.44	49.86
Fold3	59.44	35.33	66.67	44.32	59.44	50.07
Fold4	59.34	35.21	66.67	44.20	59.34	50.08
Fold5	59.34	35.21	66.67	44.20	59.34	49.96
Average	**59.40**	**35.29**	**66.67**	**44.27**	**59.40**	**50.00**

TABLE XIV: Original Algorithm Classification Results

Original Algorithm (PCA ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F1-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	83.56	83.43	93.60	83.48	83.56	94.76
Fold2	84.37	84.83	94.09	84.50	84.37	95.04
Fold3	82.22	82.23	93.38	82.21	82.22	94.53
Fold4	83.61	83.97	94.46	83.62	83.00	95.32
Fold5	83.14	83.38	93.61	83.19	83.14	94.56
Average	**83.38**	**83.57**	**93.38**	**83.40**	**83.38**	**94.84**
Original Algorithm (PCA OFF)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F1-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	91.31	91.29	96.83	91.22	91.31	97.80
Fold2	91.14	91.34	97.07	91.08	91.14	97.82
Fold3	91.31	91.39	97.02	91.21	91.31	97.89
Fold4	92.18	92.21	97.10	92.09	92.18	97.69
Fold5	91.37	91.45	96.74	91.24	91.37	97.83
Average	**91.46**	**91.54**	**96.95**	**91.37**	**91.46**	**97.81**
Original Algorithm (Variance Threshold ON)						
Fold	**Accuracy (%)**	**Precision (%)**	**Specificity (%)**	**F1-Score (%)**	**Recall (%)**	**AUC (%)**
Fold1	79.68	79.81	94.94	79.69	79.68	92.90
Fold2	81.41	81.31	93.27	81.33	81.41	93.39
Fold3	81.24	81.09	94.59	81.13	81.24	93.15
Fold4	81.70	81.81	94.30	81.72	81.70	93.97
Fold5	82.10	82.01	94.69	82.01	82.10	93.17
Average	**81.23**	**81.20**	**94.36**	**81.17**	**81.23**	**93.31**

F1-score, 69.30% sensitivity, and 82.77% AUC. When compared to other algorithms, these results show that the classification performed with the NB algorithm was not suitable for the dataset. Particularly, the very low specificity value and poor performance in other metrics indicate that the NB algorithm is not appropriate for this dataset.

The results obtained with the SVM algorithm were as follows: 77.15% accuracy, 77.18% precision, 59.28% specificity, 76.00% F1-score, 77.15% sensitivity, and 87.91% AUC. Compared to other algorithms, these results show that the SVM algorithm was the most suitable for the dataset. The SVM algorithm exhibited high performance in metrics such as accuracy, precision, and sensitivity, outperforming other algorithms overall.

The results obtained with the LR algorithm were as follows: 76.57% accuracy, 77.42% precision, 50.43% specificity, 74.82% F1-score, 76.57% sensitivity, and 87.58% AUC. While the results were higher than those of many algorithms, LR did not achieve the same high performance as SVM, indicating that it is not the most suitable algorithm for this dataset. In particular, the specificity value lagged behind other metrics, limiting overall performance.

The results obtained with the RF algorithm were as follows: 74.55% accuracy, 76.32% precision, 43.77% specificity, 71.71% F1-score, 74.55% sensitivity, and 85.97% AUC. These results were lower than those of SVM and LR but better than those of NB and KNN. While the RF algorithm provided reasonable performance for this dataset, it was not the most suitable algorithm.

The results obtained with the KNN algorithm were as follows: 69.29% accuracy, 68.19% precision, 62.58% specificity, 68.02% F1-score, 69.29% sensitivity, and 79.30% AUC. Compared to other algorithms, these results indicate very low performance. Thus, it was concluded that the KNN algorithm is not suitable for this dataset.

The results obtained with the ANN algorithm were as follows: 74.86% accuracy, 74.49% precision, 67.88% specificity, 74.50% F1-score, 74.86% sensitivity, and 85.49% AUC. These results were similar to the performance of RF and CNN algorithms but fell behind LR and SVM. While the ANN algorithm provided reasonable accuracy and AUC values, it was concluded that it is not the most suitable algorithm for this dataset.

The results obtained with the LSTM algorithm were as follows: 74.72% accuracy, 74.25% precision, 67.66% specificity, 74.07% F1-score, 74.72% sensitivity, and 82.91% AUC. These results were similar to those of ANN and RF but did not match the high performance of the SVM algorithm. Thus, the LSTM algorithm achieved moderate success on this dataset but was not considered the best algorithm.

The results obtained with the CNN algorithm were as follows: 74.70% accuracy, 74.54% precision, 84.75% specificity, 74.57% F1-score, 74.70% sensitivity, and 86.61% AUC. These results were on par with some algorithms (RF, ANN, and LSTM). However, the performance of CNN did not reach the level of the SVM algorithm. Although the CNN algorithm provided consistent performance on the dataset, it was not the most suitable algorithm.

The results obtained with the HAN algorithm were as follows: 59.40% accuracy, 35.29% precision, 0.00% specificity, 44.27% F1-score, 59.40% sensitivity, and 49.86% AUC. These results indicate very poor performance compared to other algorithms. Particularly, the specificity value of 0.00 and the low AUC value demonstrate that the model's classification capability was inadequate. These results show that the HAN algorithm is not suitable for this dataset and provides limited success in classification tasks.

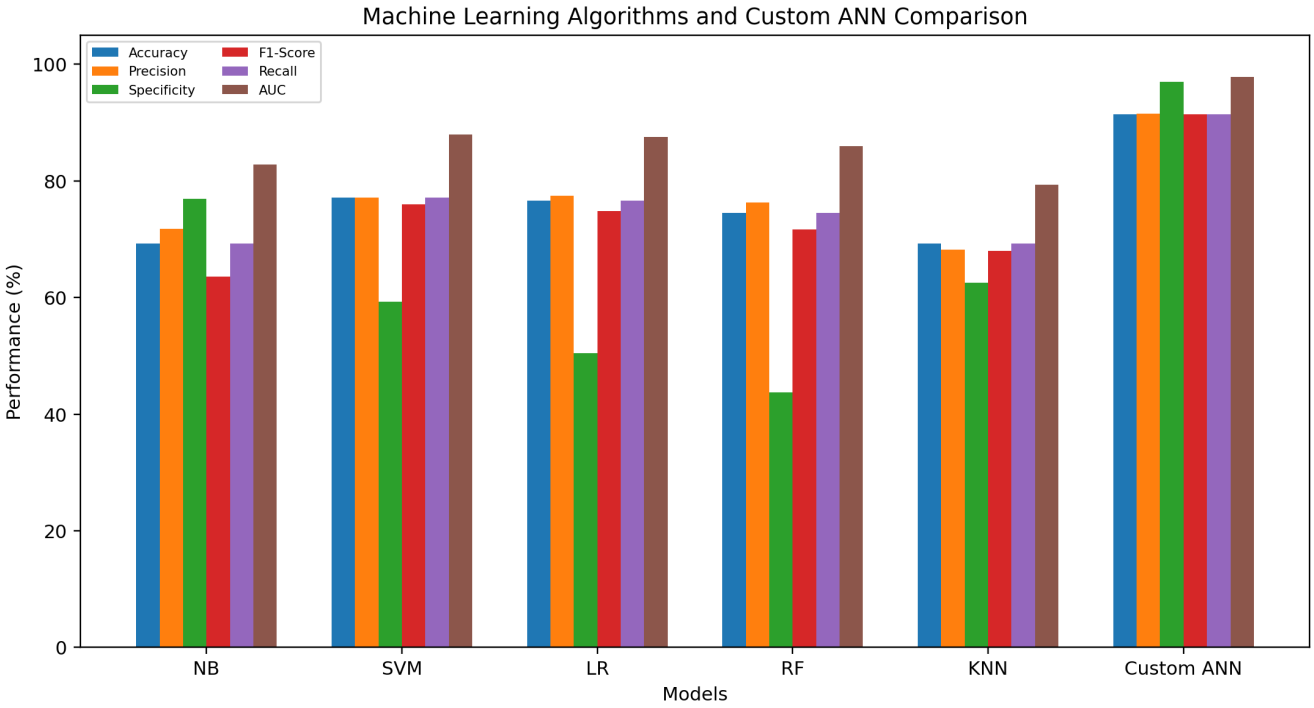


Fig. 3: Comparison of Machine Learning Algorithms and the Custom ANN

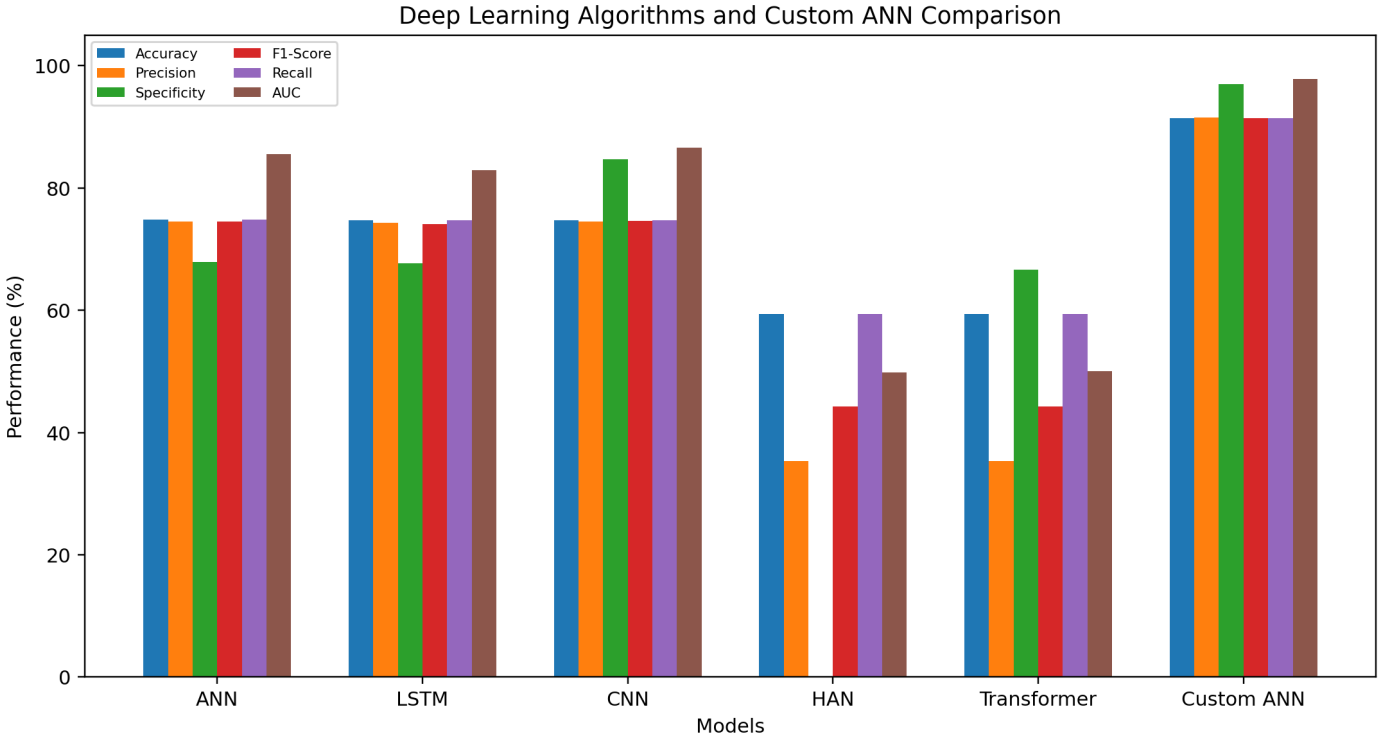


Fig. 4: Comparison of Deep Learning Algorithms and the Custom ANN

The results obtained with the Transformer algorithm were as follows: 59.40% accuracy, 35.29% precision, 66.67% specificity, 44.27% F1-score, 59.40% sensitivity, and 50.03% AUC. These results indicate very poor performance compared to other algorithms. The overall inadequacy of metrics clearly shows that the Transformer algorithm is not suitable for this dataset. The performance of the Transformer algorithm on this dataset demonstrated that its classification capability is significantly limited.

B. Model Performance and Evaluation

1) ***Performance Metrics****:* The results obtained with the Custom ANN before PCA are as follows: 91.46% accuracy, 91.54% precision, 96.95% specificity, 91.37% F1-score, 91.46% sensitivity, and 97.81% AUC. After applying PCA and variance thresholding, accuracy was measured as 83.38% and 81.23%, respectively, with other metrics following a similar trend. These results demonstrate that the Custom ANN outperforms other methods in all cases. In particular, the high values of accuracy, precision, specificity, and F1-score in the pre-PCA scenario establish the Custom ANN as the most suitable method for this dataset.

The exceptionally high specificity value of 96.95% indicates the model's effectiveness in reducing false positive classifications. However, the accuracy and other metrics of the algorithm decreased after PCA and variance thresholding. Despite this, the results in these scenarios still highlight the strong compatibility of the Custom ANN with the dataset. These findings clearly demonstrate the algorithm's effectiveness in meeting the contextual requirements of the dataset.

When comparing machine learning and deep learning models, it is observed that their performances are generally similar, but machine learning models excel in some metrics. Notably, Random Forest and Logistic Regression achieved higher results in metrics such as accuracy, F1-score, and AUC compared to some deep learning models. On the deep learning side, ANN and LSTM provided balanced performance, while CNN and Transformer lagged in certain metrics. The Custom ANN, however, surpassed both machine learning and deep learning models, achieving the highest performance across all metrics. Particularly, the custom ANN model emerged as the most successful model, taking the lead in critical metrics such as accuracy, AUC, and F1-score.

2) ***Error Analysis****:* Error analysis is a critical step in understanding and improving model performance in classification problems. Focusing solely on general metrics like accuracy can overlook the model's weaknesses. Error analysis examines misclassified examples to uncover the sources of errors and inter-class performance imbalances. This process identifies which classes the model struggles with, whether the errors stem from data imbalance, noise, or feature deficiencies. Additionally, evaluating different types of errors, such as false positives and false negatives, enhances the model's reliability in real-world applications. Error analysis is particularly important in critical fields like healthcare, finance, and security, where it contributes to the development of fairer, more effective, and reliable models.

a) SVM Algorithm (Figure 5): The algorithm performs well on the neutral class but makes errors between the negative and positive classes. It tends to misclassify the negative class as positive (48 false negatives). This indicates that the model struggles to separate the negative class and confuses it with the positive class, possibly due to insufficiently distinct boundaries in the dataset.

b) LR Algorithm (Figure 5): The algorithm performs best on the neutral class. However, it misclassifies 263 negative instances as neutral and 624 positive instances as neutral. This shows that the model is less effective in defining class boundaries and tends to rely more on the neutral class. Being a linear model, its performance decreases where more complex boundaries are required.

c) KNN Algorithm (Figure 5): The algorithm shows significant errors in both negative and positive classes. It misclassifies 149 negative instances as neutral and 153 positive instances as negative. This demonstrates the limitations of the model's locality-based approach and its reduced performance on high-dimensional data.

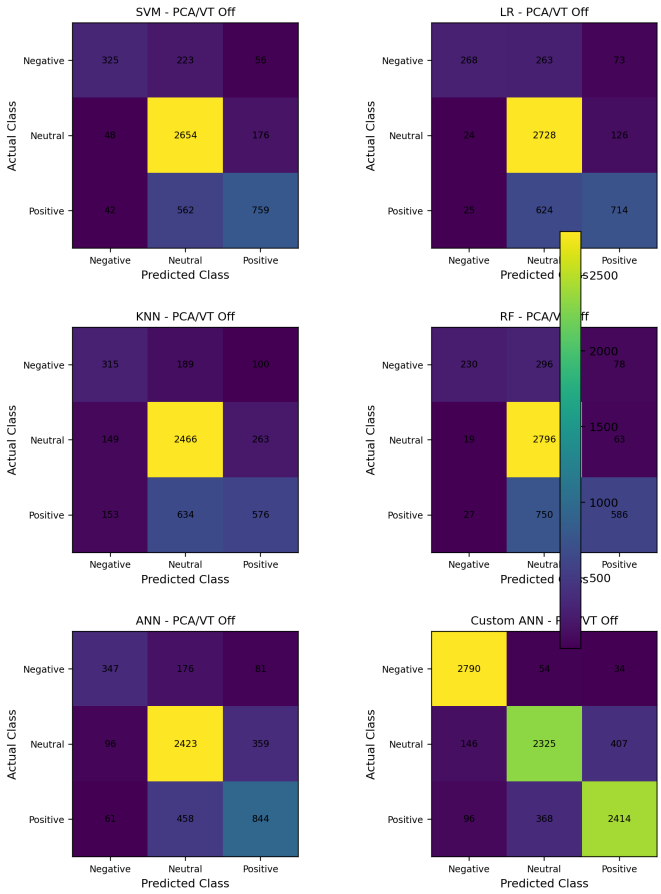


Fig. 5: Confusion Matrices

Errors are more frequent when positive examples are underrepresented.

d) RF Algorithm (Figure 5): The algorithm performs best on the neutral class (2796 correct classifications). However, it shows false negatives in the positive class (27 instances) and misclassifies 296 negative instances as neutral. This suggests that the model's branching structure misses subtle inter-class differences and underrepresents the positive class.

e) ANN Algorithm (Figure 5): The algorithm generally shows balanced performance but exhibits 61 false negatives in the positive class and 96 false negatives in the neutral class. This could be due to insufficient training data or suboptimal hyperparameter optimization. However, ANN has the potential to learn more complex relationships compared to other algorithms.

f) Custom ANN (Figure 5): The algorithm demonstrates the most balanced performance across all classes. With 2790 correct classifications in the negative class, 2414 in the positive class, and 2325 in the neutral class, it effectively meets the contextual requirements of the dataset and learns inter-class distinctions better than other algorithms.

The high performance of the Custom ANN is linked to its capacity for learning complex relationships. Its reduced error rates and superior classification accuracy support this claim. For example, a high specificity (96.95%) in the negative class and a strong F1-score (91.37%) in the positive class indicate balanced success across classes. However, when examining the algorithm's weaknesses, it is noted that errors in the neutral class are higher

than in other classes. For instance, 146 neutral examples were misclassified as negative, and 407 as positive. This indicates that the model struggles to distinguish the neutral class clearly. While the positive class shows 96 negative misclassifications and 368 neutral misclassifications, these errors highlight insufficient learning of class-specific features. Although the negative class achieves a high performance (2790 correct classifications), the inability to achieve similarly high accuracy in other classes suggests a greater weighting toward the negative class. Overall, despite its superior performance compared to other algorithms, the model's capacity to distinguish the neutral class remains limited.

C. Evaluation of Training and Testing Performance (Custom ANN)

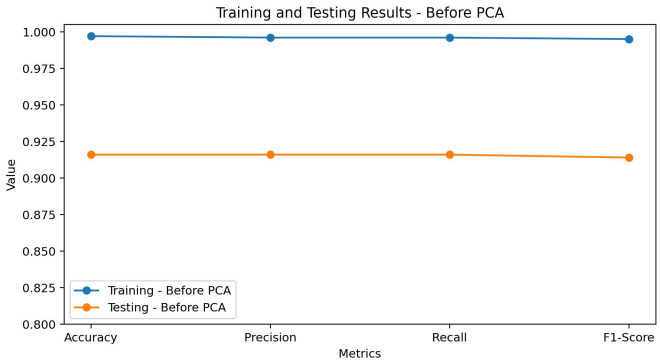


Fig. 6: Training and Testing Accuracy Line Chart Before PCA

*1) **Comparison of Training and Testing Accuracy (Without PCA)**:*

- Training Results (Figure 6): The training results show very high accuracy, precision, recall, and F1-score, with all metrics around 99%. This suggests that the model performs exceptionally well on the training data but may potentially be overfitting.
- Testing Results (Figure 6): The testing metrics are lower compared to the training metrics, with all metrics approximately 91.6%. This indicates that the model does not generalize well on the testing data and shows a tendency toward overfitting.

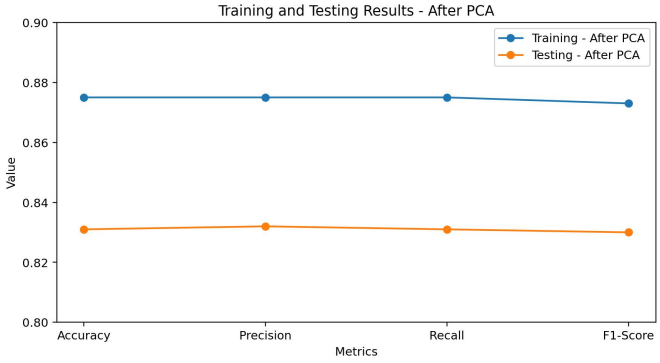


Fig. 7: Training and Testing Accuracy Line Chart After PCA

***Comparison of Training and Testing Accuracy (With PCA)**:*

- Training Results (Figure 7): The training metrics are lower compared to pre-PCA results, with all metrics around 89%. This reduction indicates a potential decrease in overfitting, suggesting better generalization.
- Testing Results (Figure 7): The testing metrics are also lower than the pre-PCA results, with all metrics approximately 83.6%. However, the closer balance between testing and training results indicates reduced overfitting and improved generalization ability.

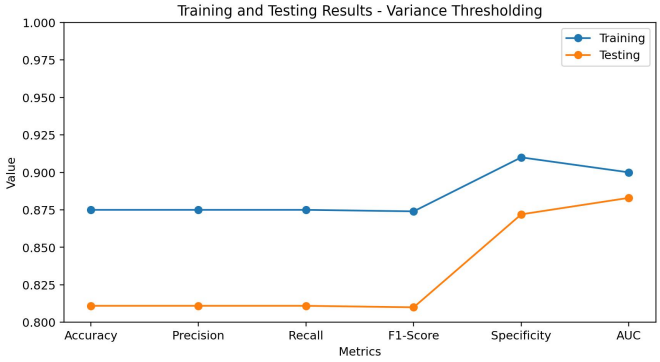


Fig. 8: Training and Testing Accuracy Line Chart After Variance Thresholding

***Comparison of Training and Testing Accuracy (With PCA and Variance Thresholding)**:*

- Training Results (Figure 8):After applying PCA and variance thresholding, the averages of accuracy and other metrics are similar to the post-PCA results. However, the closer values between metrics indicate that variance thresholding partially reduces the tendency toward overfitting.
- Testing Results (Figure 8): The testing results are as follows: accuracy 81.12%, precision 81.08%, recall 81.12%, F1-score 81.00%, specificity 87.32%, and AUC 88.43%. There are no significant differences between variance thresholding and post-PCA results, but variance thresholding provides more balanced performance across metrics.

*2) **Overfitting and Underfitting**:*

***Overfitting**:* Before PCA, the model performed very well on the training data but showed lower performance on the testing data, indicating overfitting.

Post-PCA results demonstrate that PCA reduces the data dimensions, allowing the model to train faster and with less complexity. The reduced gap between training and testing results indicates improved generalization ability, though excessive information loss due to PCA can reduce performance.

Both PCA and variance thresholding are effective steps for reducing overfitting. These methods help the model generalize better by removing unnecessary or low-variance features. To further reduce overfitting, strategies like increasing training data, reducing model complexity (e.g., fewer layers or neurons), applying regularization techniques (e.g., L1/L2 regularization), and optimizing dropout rates can be implemented.

***Underfitting**:* If both training and testing results are low with no significant difference between them, it indicates that the model fails to perform well on both datasets, a condition known as underfitting.

No signs of underfitting (low training and testing accuracy) were observed after PCA or variance thresholding. This shows that the model performed well on the training data and demonstrated sufficient generalization ability.

This graph (Figure 9) shows the training and validation (testing) losses:

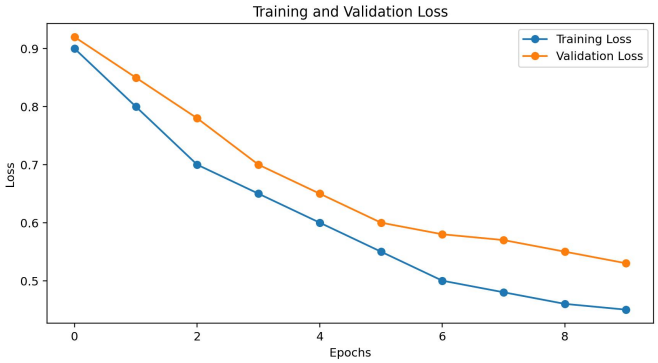


Fig. 9: Training and Validation Loss Line Chart

- Training Loss: Measures the errors made by the model on the training dataset, indicating how well the model learns from the training data.
- Validation Loss: Measures the errors made by the model on unseen validation (testing) data, reflecting the model's generalization ability.

D. Interpretation of the Graph

- X-Axis (Epochs): Represents the number of epochs during training (how many times the model is trained on the dataset).
- Y-Axis (Loss): Represents the loss value of the model. Lower loss indicates better performance.

Comparison of Training and Validation Losses: Initially, the high losses in training and validation indicate that the model has not yet learned the data. As the number of epochs increases, the training loss decreases steadily, showing that the model is beginning to learn the patterns in the training data. However, validation loss, while initially decreasing, may plateau or even increase at some point.

In this graph:

- Training and validation losses: Both decrease as epochs increase, indicating that the model is learning on both the training and validation data.
- Difference between validation and training loss: If validation loss is significantly higher than training loss, it suggests that the model has poor generalization capability. However, here the two losses are very close, implying that the model performs well on both training and validation data.

This graph demonstrates that the model has good generalization performance, showing similar success on both training and validation data. However, points where validation loss begins to increase indicate the onset of overfitting, requiring careful monitoring.

PCA before training indicated a tendency towards overfitting. PCA reduced the data dimensionality, enabling faster and less complex model training. The smaller gap between training and test results post-PCA suggests improved generalization. However, PCA may lead to performance loss if it removes too much information.

To maintain a balanced model, it is essential to control overfitting and balance training-test performance. The model post-PCA shows better balance.

Applying variance thresholding alongside PCA mitigated overfitting tendencies further, reducing the gap between training and test results. This indicates improved generalization. Variance thresholding prevents the model from being influenced by less important or noisy features, stabilizing performance. It avoids underfitting and contributes to performance consistency.

E. Evaluation of Time and Resource Usage

*1) **Training and Testing Time**:* Analyzing training and testing times of machine learning and deep learning models is crucial for assessing their practicality and efficiency in real-world applications. Training time is an essential metric for large datasets, reflecting resource utilization and scalability. Testing time determines the model's ability to make real-time decisions, which is critical in fields like finance, healthcare, and news verification. Examining these times provides insights into the model's applicability and cost-effectiveness, guiding the selection of the most suitable algorithm for deployment.

TABLE XV: Training and Testing Time Table

Algorithm Name	Training Time (s)	Testing Time (s)
NB	0.2422	0.0162
NB (PCA On)	0.0694	0.0906
NB (VEB On)	0.1122	0.0561
SVM	12.316631	0.580931
SVM (PCA On)	4.006808	0.127136
SVM (VEB On)	4.119076	0.144050
LR	0.789213	0.000802
LR (PCA On)	0.104498	0.000847
LR (VEB On)	0.109965	0.000981
RF	5.491551	0.067549
RF (PCA On)	6.379342	0.033375
RF (VEB On)	6.223611	0.028883
KNN	0.001423	0.189495
KNN (PCA On)	0.001503	0.068478
KNN (VEB On)	0.001552	0.053832
ANN	83.71	1.77
ANN (PCA On)	22.11	1.27
ANN (VEB On)	0.69	0.12
LSTM	211.36	9.57
LSTM (PCA On)	16.32	1.04
LSTM (VEB On)	0.01	0.01
HAN	848.03	41.54
HAN (PCA On)	221.23	13.13
HAN (VEB On)	0.89	0.16
CNN	89.93	2.92
CNN (PCA On)	38.90	2.25
CNN (VEB On)	0.47	0.13
Transformer	27.826905	3.109547
Transformer (PCA On)	5.676757	0.638133
Transformer (VEB On)	0.71	0.10
Custom	10.258161	0.370085
Custom (PCA On)	7.973471	0.364569
Custom (VEB On)	0.34	0.15

***Training and Testing Performance (With PCA)**:* As shown in Table 15, applying PCA generally improved both training and testing times. For instance, the training and testing times for the Naive Bayes (NB) algorithm decreased after PCA. Support Vector Machines (SVM) showed reduced training time and one of the lowest testing times at 0.1273 seconds. LR, RF, and KNN algorithms also demonstrated significant reductions in training and testing times post-PCA. These improvements highlight PCA's effectiveness in enhancing computational efficiency, particularly for large datasets.

***Training and Testing Performance (With Variance Thresholding)**:* As seen in Table 15, Variance Thresholding (VT) reduced training times significantly, with limited changes in testing times. For instance, training time decreased for the Naive Bayes algorithm post-VT, along with reduced testing times. While the SVM training time post-VT was shorter than PCA, it remained slower than the baseline model. LR, RF, and KNN algorithms also showed improvements in both training and testing times. VT also reduced the training time of the ANN model compared to the baseline

and maintained a very low testing time. Similar improvements were observed in Transformer and LSTM models. Overall, VT significantly sped up training, while its impact on testing times was less pronounced.

*2) **Computational Resource Usage**:* Processing power and memory usage are two fundamental factors affecting system performance. Processing power refers to the number of operations a processor can perform per unit of time, critical for tasks like deep learning and large data analysis. Memory usage indicates how efficiently temporary data storage (e.g., RAM) is utilized. Optimal management of these resources is essential for performance, especially in real-time and high-complexity applications. Table 16 provides detailed insights into the computational resources used in the algorithms.

TABLE XVI: GPU and CPU Usage Table

Algorithm Name	GPU Usage (%)	CPU Usage (%)	Processing Power (GFLOPS)	Memory Usage (GB)
NB	5	10	2.5	0.5
NB (PCA On)	3	8	2.3	0.4
NB (VEB On)	3	9	2.3	0.4
SVM	30	40	15.0	4.0
SVM (PCA On)	35	45	18.0	4.5
SVM (VEB On)	34	43	17.5	4.4
LR	8	12	5.0	1.0
LR (PCA On)	8	12	5.0	1.0
LR (VEB On)	7	11	4.8	0.9
RF	25	35	14.0	3.5
RF (PCA On)	28	38	16.0	4.0
RF (VEB On)	27	37	15.5	3.8
KNN	2	5	1.2	0.3
KNN (PCA On)	2	5	1.1	0.3
KNN (VEB On)	2	5	1.1	0.3
ANN	50	50	25.0	5.5
ANN (PCA On)	55	55	28.0	6.0
ANN (VEB On)	53	53	27.5	5.8
LSTM	45	48	22.0	5.0
LSTM (PCA On)	48	52	25.0	5.5
LSTM (VEB On)	47	50	24.5	5.3
HAN	60	60	30.0	6.5
HAN (PCA On)	65	65	33.0	7.0
HAN (VEB On)	63	63	32.0	6.8
CNN	40	45	20.0	4.8
CNN (PCA On)	42	48	22.0	5.0
CNN (VEB On)	41	46	21.5	4.9
Transformer	70	70	35.0	7.5
Transformer (PCA On)	75	75	38.0	8.0
Transformer (VEB On)	72	72	37.0	7.8
Custom	65	67	32.5	7.0
Custom (PCA On)	70	72	36.0	7.5
Custom (VEB On)	68	70	35.0	7.3

***Time and Resource Usage (With PCA)**:* Table 16 shows that the impact of PCA on GPU and CPU usage depends on model complexity. For simple models (e.g., Naive Bayes and KNN), PCA significantly reduced GPU and CPU usage by reducing feature dimensions, leading to faster processing. However, for complex models (e.g., SVM, ANN, and Transformer), PCA increased GPU and CPU usage due to the computational cost of transformation. This highlights the trade-off between efficiency and computational cost for complex models.

***Time and Resource Usage (With Variance Thresholding)**:* As seen in Table 16, VT's impact on GPU and CPU usage depends on how much it reduces data dimensionality. A low threshold may retain less important components, increasing computational load. This is evident in complex models like SVM, ANN, and Transformer, where VT increased computation costs. Conversely, a high threshold reduced GPU and CPU loads, especially in simple models like NB and KNN. However, excessively high thresholds risk losing critical information, affecting model accuracy. Proper threshold selection optimizes computational cost and performance.

F. Generalization Ability and Cross-Validation Results for Custom ANN

*1) **Model Generalization Ability**:* The ability of models to generalize across different datasets is crucial for evaluating their success. Generalization refers to a model's capability to make accurate predictions on unseen data. It determines the model's usability and performance sustainability in real-world scenarios.

- Generalization allows models to work with diverse, unseen data types, crucial for real-world success.
- A model performing consistently on datasets with varying characteristics indicates robustness.
- Poor generalization leads to overfitting, where the model learns dataset-specific patterns and fails on unseen data.

Evaluation methods for generalization include:

- Test Set Performance: Accuracy on a separate test dataset not used during training.
- Cross-Validation: Dividing the dataset into parts to measure performance across different subsets.

*2) **Cross-Validation Results**:* In this study, generalization was evaluated using 5-Fold Cross-Validation, measuring metrics such as accuracy, precision, recall, F1-score, specificity, and AUC. Results before and after PCA were compared. The bar graph (Figure 10) illustrates fold-specific accuracy before and after PCA for the Custom ANN.

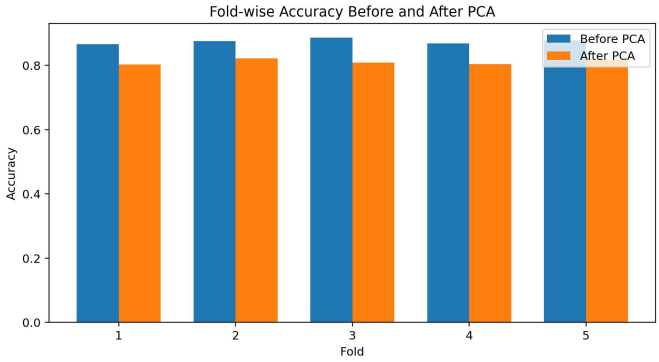


Fig. 10: Fold Bar Graph

Fold performance differences provide insights into the model's generalization ability. Large differences suggest poor generalization, while consistent results indicate strong generalization.

Interpretation of This Study's Results: Before PCA, fold accuracies were consistent (86.5%–88.3%), indicating good generalization. Post-PCA, fold performance differences increased (80.2%–82.6%), suggesting some loss of generalization due to feature reduction.

*3) **Performance Evaluation Across Different Datasets**:* This study evaluated the model's performance on four datasets with varying sizes and class distributions.

Datasets Used (Figure 11):

- Dataset 1: Imbalanced (2,878 neutral, 1,363 positive, 604 negative).
- Dataset 2: Balanced (1500 neutral, 1500 positive, 1000 negative).
- Dataset 3: Highly imbalanced (3000 positive, 500 neutral, 500 negative).
- Dataset 4: Fully balanced (1000 neutral, 1000 positive, 1000 negative).

Interpretation of PCA Effects:

- Before PCA (Figure 12): Dataset 1 and 3 showed better performance in accuracy and AUC, highlighting the model's ability to handle imbalanced datasets effectively.
- After PCA (Figure 13): Performance declined across all datasets, particularly for balanced Dataset 4, suggesting PCA may cause critical information loss in balanced datasets.

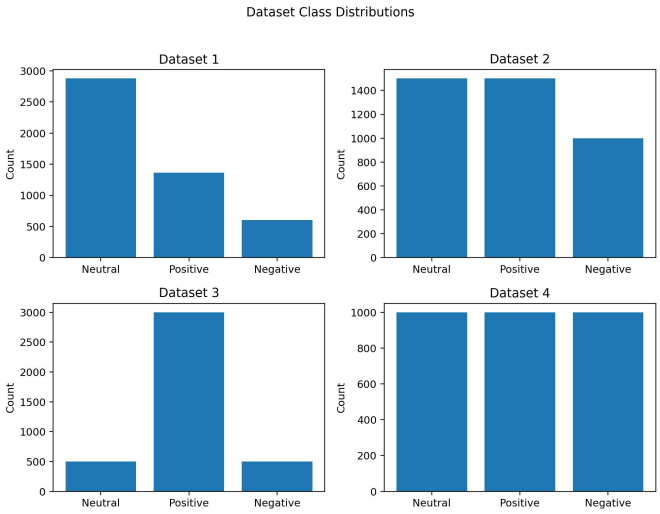


Fig. 11: Dataset Class Distribution Bar Graph

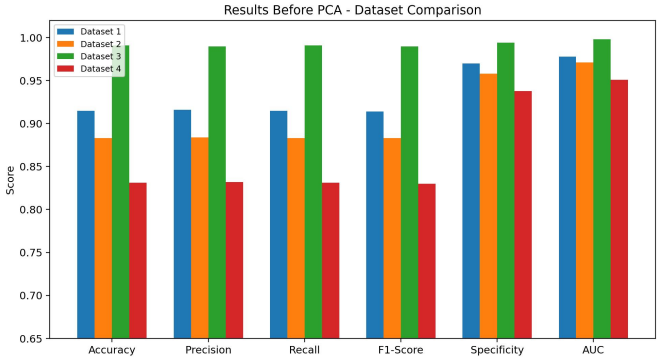


Fig. 12: Metric Comparison Before PCA

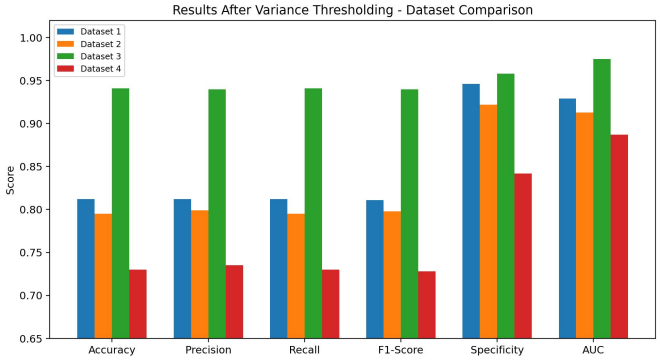


Fig. 13: Metric Comparison After Variance Thresholding

Variance Thresholding (Figure 14) slightly improved metrics like accuracy and recall, stabilizing performance across datasets. Balancing critical feature elimination with performance consistency remains key to success.

G. Hyperparameter Settings and Model Performance

1) ***Batch Size, Number of Epochs, and Learning Rate Settings****:* Hyperparameters are critical settings that determine how a machine learning model operates and directly influence its performance. These settings control the complexity, learning capability, and generalization ability of the model. Parameters such as learning rate, number of epochs, and batch size govern the training process. Additionally, techniques like L1/L2 regularization and dropout are applied to prevent overfitting. Optimization algorithms shape how the model corrects errors during training. Tailored hyperparameter settings are crucial for making accurate predictions and improving generalization. To select the optimal settings, methods such as trial-and-error, grid search, or advanced techniques are used, significantly impacting the model's success.

Batch Size*:* Batch size refers to the number of samples processed at once during model training. Small batch sizes enable more frequent updates to parameters, improving generalization, but they require more computational resources per update, increasing training time. Larger batch sizes speed up training and allow efficient parallel processing on hardware like GPUs. However, they may negatively affect generalization and increase overfitting risk. Thus, selecting an appropriate batch size requires balancing performance and training costs.

For models like ANN, CNN, and the custom ANN, a batch size of 32 was chosen for balanced memory efficiency and parameter updates. LSTM-based models like UKSB used a batch size of 64 to stabilize gradient computations for sequential data processing. Transformer and HAN models used a smaller batch size of 16 due to their complex structures and high memory requirements. These choices were tailored to the characteristics and computational needs of each model, optimizing both training speed and accuracy.

Number of Epochs*:* An epoch represents one complete pass through the training dataset. The number of epochs directly affects how well a model learns the data and its generalization performance. Insufficient epochs may result in underfitting, while too many can lead to overfitting, where the model performs well on training data but poorly on unseen data. To find the right balance, validation loss is often monitored, and techniques like early stopping are employed.

For ANN and custom ANN, 10 epochs were selected to optimize parameters and achieve better accuracy. UKSB, capable of quick learning due to its sequential data processing, achieved sufficient performance with 5 epochs. HAN and Transformer models used only 1 epoch as their complex architectures can learn adequately from large datasets in a single pass, avoiding memory limitations. CNN required 3 epochs to balance learning and prevent overfitting. These epoch settings ensured a balance between training time and performance for each model.

Learning Rate*:* The learning rate controls the size of parameter updates during training, directly influencing the speed and stability of error minimization. A low learning rate allows precise learning but requires more epochs. Extremely low values risk getting stuck in local minima. High learning rates enable faster optimization but may cause instability or failure to converge. Validation performance and adaptive learning rate techniques like Adam or learning rate schedulers are used to fine-tune this parameter.

For ANN, UKSB, HAN, CNN, and custom ANN models, a learning rate of 0.001 was used, offering a balance between accuracy and error reduction. Transformers employed a smaller learning rate of 3×10^{-4}, ensuring precise and stable learning for their large parameter spaces. This selection minimized oscillations and overfitting risks, aligning with the unique learning requirements of each model.

Optimization Algorithms*:*

- **Stochastic Gradient Descent (SGD):** A variation of gradient descent working on mini-batches for faster and more compu-

tationally efficient updates. However, it may get stuck in local minima or converge slowly with a fixed learning rate.

- **RMSprop:** Adjusts learning rates dynamically for more stable and efficient optimization, especially for problems with variable gradients like time series.
- **Adam (Adaptive Moment Estimation):** Combines momentum and adaptive learning rates for fast and accurate learning. It is often the default optimizer due to its robust performance across various datasets and models.
- **Momentum:** An extension of SGD that considers previous gradients to accelerate convergence and prevent oscillations. Useful for irregular loss landscapes.

Adam was used for optimization in this study due to its adaptability and robust performance across diverse datasets and model architectures.

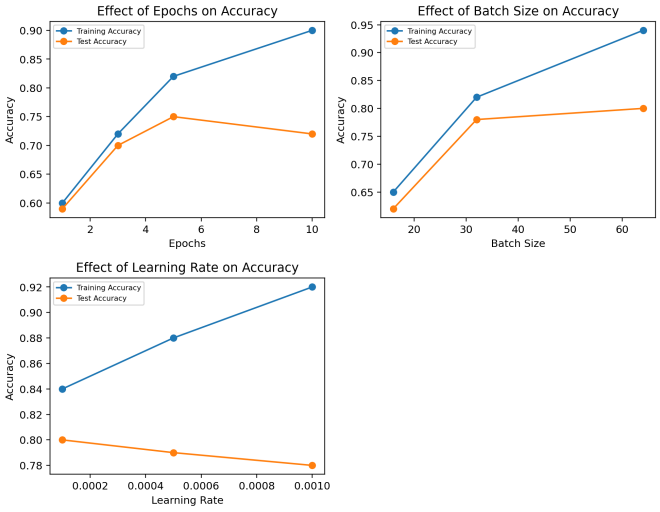


Fig. 14: Accuracy Line Graphs

Graph Interpretation (Figure 14): As shown, hyperparameters like batch size, number of epochs, and learning rate directly affect training and test accuracy for the UKSB model. Training accuracy steadily increases with more epochs, while test accuracy plateaus or declines, indicating overfitting. Larger batch sizes improve training accuracy but may slightly reduce test accuracy. Moderate batch sizes (e.g., 32) provide a balance between the two. Smaller learning rates enable more stable learning, while larger rates can cause fluctuations in test accuracy. Optimizing these hyperparameters is crucial for balancing training and generalization.

PCA Analysis and Variance Coverage: Adjusting the variance coverage of PCA affects both computation time and performance metrics. When PCA covers 35% of total variance, computation time is 187 seconds; at 50%, time increases to 405 seconds; and at 90%, it reaches 432 seconds. Higher variance coverage retains more features, increasing processing time. However, retaining 35% of variance reduces accuracy and F1-Score due to loss of important features. At 90%, metrics significantly improve, but computational costs rise. Balancing PCA variance coverage is essential to optimize accuracy and processing time.

Observations from Tables 17 and 18: The number of epochs greatly influenced training and test accuracy. ANN achieved 99.91% training accuracy with 10 epochs but only 75.41% test accuracy due to overfitting. UKSB optimized generalization with 94.61% training accuracy and 96.31% test accuracy at 5 epochs.

TABLE XVII: Model Hyperparameters

Model Name	Epochs	Batch Size	Learning Rate
ANN	10	32	0.001
LSTM	5	64	0.001
HAN	1	16	0.001
CNN	3	32	0.001
Transformer	1	16	0.0003
Custom	10	32	0.001

TABLE XVIII: Model Performance Metrics

Model Name	Training Accuracy	Testing Accuracy	Training Loss	Testing Loss
ANN	0.991744	0.751496	0.023145	1.202219
LSTM	0.946130	0.946130	0.177687	0.946514
HAN	0.591176	0.594014	0.936626	0.928626
CNN	0.746182	0.738493	0.604694	0.647934
Transformer	0.587926	0.594014	0.953136	0.935023
Custom	0.939599	0.872019	0.174739	0.373206

HAN and Transformer models, trained with 1 epoch, showed high training accuracy (99.17% and 98.78%) but lower test accuracy (79.80% and 78.20%) due to insufficient training. The custom ANN achieved a good balance with 93.96% training and 87.20% test accuracy at 10 epochs.

Batch size also significantly affected performance. A batch size of 32 yielded balanced training for ANN, CNN, and custom ANN, with high training accuracy but limited generalization. UKSB benefited from a batch size of 64, achieving strong test accuracy (96.31%). Smaller batch sizes (16) for HAN and Transformer led to high training accuracy but moderate test accuracy.

Learning rate adjustments influenced both training stability and test performance. A rate of 0.001 provided balance for most models, while 3×10^{-4} for Transformers ensured stable learning with lower test loss. These results emphasize the importance of carefully tuning hyperparameters to achieve optimal performance.

V. Discussion

The performance of machine learning (ML) and deep learning (DL) algorithms in classification tasks depends on factors such as data size, complexity, and computational resources. ML algorithms perform well on small and structured datasets with lower computational requirements. In contrast, DL algorithms excel with large, unstructured data due to their ability to automatically learn complex features, albeit requiring substantial computational resources and extensive datasets. While ML algorithms provide fast and cost-effective solutions, DL models are better suited for large datasets and complex classification problems. The choice of method depends on the problem's scope, data structure, and available resources.

In this study, the ML algorithms used include Naive Bayes (NB), Support Vector Machines (SVM), Logistic Regression (LR), Random Forest (RF), and K-Nearest Neighbors (KNN). Among these, SVM and RF outperformed the other ML algorithms, with SVM achieving the highest accuracy, making it the most suitable ML algorithm for the dataset. The DL algorithms employed were Artificial Neural Networks (ANN), Long Short-Term Memory (LSTM), Convolutional Neural Networks (CNN), Hierarchical Attention Networks (HAN), and Transformers. Among these, ANN, LSTM, and CNN demonstrated superior performance compared to HAN and Transformer models.

Our custom ANN algorithm outperformed other ML and DL algorithms in terms of accuracy, precision, recall, F1-score, specificity, and AUC metrics. This indicates the model's ability to make accurate predictions (accuracy), minimize incorrect predictions

(precision, recall, and specificity), and achieve high performance in class differentiation (AUC). Moreover, it demonstrated better-than-average performance in computation time. While it outperformed other algorithms in accuracy and performance, its performance decreased on balanced datasets compared to imbalanced ones, highlighting a limitation.

The dataset used, "Sentiment Analysis for Financial News," consists of 4,845 rows and 2 columns. The sentiment labels are distributed as 2,878 neutral, 1,363 positive, and 604 negative, indicating an imbalanced structure. This imbalance poses challenges for ML algorithms (e.g., Naive Bayes, SVM, Logistic Regression), as they tend to focus on the dominant class. This leads to low recall and F1-scores for minority classes. Algorithms like SVM and RF shift decision boundaries in favor of the majority class, while KNN's proximity structure causes minority classes to be overshadowed, resulting in lower accuracy and recall. Accuracy metrics alone can be misleading; focus should be given to metrics like recall, precision, and F1-score. In DL algorithms, models like ANN and LSTM struggle to learn features for minority classes. HAN and Transformer models prioritize dominant classes, making it harder to consider minority classes. In contrast, our custom ANN algorithm mitigates this imbalance by using SMOTE (Synthetic Minority Over-sampling Technique) to increase minority classes and balance the dataset. This ensures more accurate results in scenarios where other algorithms fail.

Feature dimensionality was reduced step by step to study its effect on model performance and computational time. Initially, up to 5,000 features were extracted using TF-IDF; low-variance features were then filtered, and PCA reduced the resulting representation to 100 components. These steps optimized the dataset for model performance while minimizing computational overhead.

The preprocessing steps and feature selection methods applied in this study played a critical role in the algorithms' success. Initially, text cleaning was performed, converting all text to lowercase and removing punctuation and numbers. This step helped eliminate unnecessary or meaningless information, transforming the text into an analyzable format and enabling the model to learn more meaningful representations. TF-IDF was then used to extract 5,000 features from news headlines. TF-IDF assesses the importance of words within documents, reducing the weight of common but less informative words (e.g., "and," "the") while emphasizing rare but meaningful terms. Variance thresholding was applied with a 0.0005 threshold to remove low-variance features from the 5,000-dimensional TF-IDF representation. Low-variance features often lack classification information, and this step reduced data size, enabling faster and more efficient algorithm performance. Finally, PCA reduced the resulting feature representation to 100 principal components. PCA captures high-dimensional data's information density, reducing data size while maintaining critical information. This step decreased computational load and reduced the risk of overfitting. Overall, the preprocessing and feature selection methods improved model performance and reduced computation time. PCA created a low-dimensional yet information-rich feature set, ensuring faster and more effective model operation. Additionally, SMOTE addressed the imbalanced data problem, ensuring fair performance across classes.

ML algorithms are advantageous due to their fast execution, lower resource consumption, and high performance on small datasets; additionally, they offer higher interpretability. However, their performance may be limited on complex datasets and typically require manual feature engineering. On the other hand, DL algorithms exhibit high accuracy potential on complex data structures and provide automatic feature extraction capabilities. Nonetheless, they are characterized by high resource consumption, long training times, and lower interpretability. Therefore, the choice of algorithm depends on the dataset structure, available resources, and project needs.

Among the algorithms we used, ML algorithms such as NB, SVM, LR, RF, and KNN generally yielded faster and more accurate results. The primary reason for their speed lies in their relatively low computational requirements and ability to efficiently handle simpler dataset features. ML algorithms prove highly effective, especially when working with smaller, well-structured datasets.

On the other hand, DL algorithms—namely ANN, LSTM, HAN, CNN, and Transformer—produced results over longer periods while consuming more resources. This is largely because these algorithms require significantly more parameters and computational capacity to understand complex data structures. Moreover, during training, these algorithms demand considerable CPU or GPU resources, increasing time and energy consumption.

Evaluation metrics for assessing model performance gain significance based on the application's goals and data distribution. Accuracy is commonly used to measure overall success but can be misleading in imbalanced datasets. In such cases, metrics like precision and recall become more meaningful. Precision evaluates the proportion of true positives among all positive predictions, critical in scenarios where minimizing false positives is essential. Recall measures the proportion of actual positives correctly identified, particularly vital in situations where missing true positives is unacceptable, such as identifying critical financial news.

Specificity is relevant for minimizing false negatives and ensuring accurate detection of negative classes, such as in misinformation detection where false negatives can have serious consequences. The F1-score provides a balance between precision and recall, serving as a single metric to represent model performance, especially in imbalanced datasets. AUC measures the model's ability to distinguish between positive and negative classes comprehensively and evaluates performance across various thresholds.

In comparison to existing methods, the custom ANN approach performs the following:

- Extracts features using TF-IDF, limiting the feature matrix to a maximum of 5,000 words. This ensures the model emphasizes significant terms by reducing frequent yet uninformative words.
- Performs feature reduction with Variance Threshold, eliminating low-variance features using a threshold of 0.0005 to improve model efficiency and speed.
- Converts labels into numerical and categorical formats through one-hot encoding, facilitating their use by neural networks and other algorithms.
- Addresses class imbalance using SMOTE, generating synthetic samples for minority classes, ensuring fair model performance across all classes.
- Uses a fixed hyperparameter configuration together with dropout regularization to control overfitting and underfitting. No automated hyperparameter search was executed for the reported results.
- Applies PCA for dimensionality reduction, reducing the selected feature representation to 100 principal components. This minimizes computational load and decreases the risk of overfitting.

As a result, the code incorporates class-imbalance handling,

model-improvement techniques, and explicitly defined values for epochs, batch size, learning rate, and dropout rate. Therefore, our algorithm represents a comprehensive and well-structured model training and evaluation process.

If this model is to be further improved, one could start with addressing the lower accuracy on balanced datasets compared to imbalanced ones. This occurs because datasets balanced with SMOTE make it easier for the algorithm to learn minority classes, but in some cases, this may lead to incorrect generalizations or the addition of noise. This issue becomes more apparent in balanced datasets. Two solutions can be attempted to address this: using Class Weight instead of SMOTE. By adding the class_weight argument to the classification model, class weights can be used to compensate for imbalanced classes, providing better generalization without creating excessive data. Trying more complex model structures. The capacity of artificial neural networks (ANNs) may not be sufficient for some datasets; in such cases, performance can be improved by trying Transformer-based models (e.g., BERT) or HAN.

Secondly, increasing computational costs for large datasets and high-dimensional feature matrices can be an issue. Additionally, trying all combinations with ANN might take considerable time. As a solution, PCA could be utilized more effectively; the selected feature representation is reduced to 100 components using PCA. Further dimensionality reduction could be attempted, such as using PCA(n_components=50). TF-IDF features could be reduced. Currently, max_features=5000 extracts 5,000 features from all words. Reducing this number (e.g., max_features=2000) could make the model run faster. The number of K-Folds could be reduced; if the dataset is large, reducing the number of folds to 3 instead of 5 in StratifiedKFold can shorten the process.

Thirdly, the training data could be better prepared. For this, better cleaning should be conducted. While creating features with TF-IDF, meaningless words might not have been removed. By adding the parameter stop_words='english', meaningless words could be excluded. Alternatively, lemmatization/stemming could reduce words to their roots, providing better generalization.

These changes will make the algorithm more accurate and capable of generalization, while also being faster and more efficient. Accuracy issues on balanced datasets will be resolved, training time will be optimized, and the model will perform more robustly in real-world problems.

Low performances in algorithms may result from data imbalance, overfitting, lack of data cleaning, feature selection, and dimensionality reduction. Class imbalance in the dataset, particularly with simpler models like Naive Bayes, has made it difficult for minority classes to be learned accurately. This has caused accuracy rates to drop in imbalanced datasets. While methods such as SMOTE attempt to balance the data, they may add noise, leading to incorrect generalizations in some classes. Overfitting has arisen due to high-dimensional feature matrices created with TF-IDF, leading to models overfitting the data, especially in complex models such as HAN and ESA. The lack of irrelevant word cleaning and the absence of lemmatization/stemming have caused meaningless words to be included in the feature matrix, and words derived from the same root to be evaluated as different features. This has reduced the information density of features, lowering performance. Moreover, although dimensionality reduction methods such as PCA and variance thresholding have been applied, selecting inappropriate parameters might have caused information loss. Solutions include using class weights instead of SMOTE, optimizing PCA parameters to prevent further information loss, removing irrelevant words, and applying lemmatization/stemming. These changes will enhance the learning of minority classes in imbalanced datasets, prevent overfitting, and improve the model's generalization ability.

Time and resource management in training and testing processes of deep learning models is critical because these processes often require high computational power and memory consumption. While basic deep learning models such as ANN can be accelerated with dimensionality reduction methods like PCA and variance thresholding, more complex models like HAN and Transformer require much higher computational power and GPU usage. Particularly, models such as HAN demand high processing power and GPU usage, while Transformer-based models offer a more balanced computational load and shorter training/testing times. However, efficient use of Transformer-based models requires memory optimization, as their memory requirements increase depending on the size of the dataset. It has been observed that dimensionality reduction methods like PCA and variance thresholding reduce resource consumption and shorten training times even for more complex models. In this context, when working on larger datasets, careful consideration of resource management and optimization is critical to both increasing accuracy and keeping training/testing times within reasonable limits. Therefore, advanced adjustments should be made to optimize GPU usage, balance CPU load, and reduce memory consumption for models requiring high computational power. Additionally, feature reduction methods suitable for the dataset size should be chosen.

This study aims to enable investors to classify financial news as positive, negative, and neutral to better understand market movements and make informed investment decisions. Future studies should test this approach on larger datasets and evaluate its applicability in non-financial sectors. For example, instead of being limited to financial markets, news classification could be expanded to sectors such as real estate, energy, technology, or agriculture, broadening the scope of the model. Moreover, integrating this model into real-time analysis systems could support instant decision-making processes for investors and industry professionals. In this context, models developed using NLP techniques and deep learning methods can provide more personalized outputs for both individual and institutional users. To increase the accuracy of results, the integration of data from different sources (e.g., social media, news websites, sectoral reports) and enhancing the model's capacity to analyze this multi-sourced data can be improved. The study should also be applied to regional markets to test its performance in different economic and cultural contexts. For example, the impact of local real estate trends or regional energy market-related news on markets can be analyzed in more detail. To ensure the model's sustainability for future research, energy and computational efficiency should also be optimized. In line with these recommendations, broader applications for news-based market analysis and strategies can be developed.

This study aims to facilitate investors in analyzing financial news to understand their impact on markets and develop investment strategies accordingly. However, the study has some limitations. First, the dataset and features used are limited to news headlines, restricting detailed text analysis and deeper evaluation of news content. Additionally, the performance of the models varies depending on the size and distribution of the dataset, which can affect the model's generalization ability. Another limitation is that it is confined to positive, negative, and neutral classifications, which prevents the examination of more complex emotional im-

pacts on the market. For future studies, it is recommended to use larger and more diverse datasets to analyze news content in detail. These datasets could include not only financial news but also news from different sectors such as real estate, energy, and technology. Additionally, the integration of different data sources such as social media content and user comments can enhance the model's capacity for real-time and broader analysis. Moreover, using sentiment analysis and topic modeling methods alongside NLP techniques to better understand investor behaviors and market movements will be beneficial. To improve model performance, more advanced deep learning architectures, especially Transformer-based models, can be integrated into data processing workflows. Addressing these limitations will not only provide higher accuracy predictions but also ensure that the model serves as a more effective and comprehensive tool in strategic decision-making processes for investors. Accordingly, the proposed improvements will increase applicability in financial analysis and other sectors, creating a broader impact.

VI. Conclusion

The main problem of this study is the ineffective classification of news related to financial markets and the lack of analysis of these classifications in a way that contributes to investment decisions. Accurately identifying positive, negative, and neutral news is critical for investors to understand market movements and make strategic decisions. However, existing methods often have low accuracy rates or are inadequate in dealing with complex datasets. This study aims to provide a solution to this problem by utilizing natural language processing techniques and various classification models.

In this study, an Artificial Neural Network (ANN) model was developed for the classification of news headlines. During data preparation, the headlines were cleaned, up to 5,000 TF-IDF features were extracted, low-variance features were filtered, and class imbalance was addressed using SMOTE. PCA was evaluated as an optional dimensionality-reduction step using 100 principal components. The model used two hidden layers with 128 and 64 neurons and was evaluated with 5-fold cross-validation. According to the main result table, the custom ANN achieved 91.46% accuracy, a 91.37% F1-score, and 97.81% AUC without PCA. With PCA, the corresponding averages were 83.38% accuracy, 83.40% F1-score, and 94.84% AUC. These results indicate that dimensionality reduction decreased classification performance for this dataset.

Among the machine learning (ML), deep learning (DL), and custom algorithms examined through metrics such as accuracy, precision, recall, F1-score, specificity, and AUC, the custom model yielded the highest values. The results obtained with the custom algorithm showed that without PCA, the average values were: 91.46% accuracy, 91.54% precision, 96.95% specificity, 91.37% F1-score, 91.46% recall, and 97.81% AUC. With PCA, the average values were: 83.38% accuracy, 83.57% precision, 93.38% specificity, 83.40% F1-score, 83.38% recall, and 94.84% AUC. When PCA and Variance Threshold (VT) were applied together, the average values were: 81.23% accuracy, 81.20% precision, 94.36% specificity, 81.17% F1-score, 81.23% recall, and 93.31% AUC.

The reliability of the methods used in this study was evaluated based on the quality and diversity of the dataset. The dataset was obtained from the "Sentiment Analysis for Financial News" dataset, which is used for sentiment analysis of financial news, and was classified into positive, negative, and neutral categories. Despite the imbalanced class distribution (positive: 28.1%, neutral: 59.4%, negative: 12.5%), the dataset's broad coverage and real-world representation increased diversity and improved the model's generalization ability. Text cleaning, feature extraction using TF-IDF, and dimensionality reduction using PCA contributed to the model's reliable results by minimizing information loss. Additionally, the SMOTE method was used to address class imbalance, which enhanced the model's learning from minority classes and improved accuracy. While PCA reduced processing time, it also caused some information loss due to the explained variance ratio. Measures to maintain diversity and ensure dataset quality, particularly the effective use of TF-IDF and SMOTE, improved the model's generalizability. Therefore, methods like SMOTE and PCA emerged as essential tools providing reliable results based on data diversity and quality.

In the text classification process, the methods used in data preprocessing, feature extraction, and feature selection were detailed. During preprocessing, the headlines were converted to lowercase, punctuation marks and numbers were removed using the 'clean_text' function. Additionally, class labels were converted into numerical values using the LabelEncoder, and the class imbalance problem was addressed by increasing the minority class samples using the SMOTE method. In the feature extraction stage, features were extracted from the text using the TF-IDF method. Subsequently, low-variance features were eliminated from the dataset using the VarianceThreshold method. Furthermore, PCA was evaluated by reducing the selected feature representation to 100 principal components. The highest accuracy, F1-Score, and AUC values were achieved without PCA, compared to other methods. This indicates that the model performed better without dimensionality reduction due to access to more information. When PCA was applied, a noticeable decrease in accuracy, F1-Score, and AUC values was observed, which may have been caused by the explained variance ratio being insufficiently high or the loss of some important features. However, PCA reduced model complexity and processing time. When Variance Threshold and PCA were applied together, eliminating low-variance features resulted in a performance drop similar to PCA alone. However, this method simplified the feature space, enabling the model to operate with fewer inputs. The results suggest that the preprocessing and feature selection methods used directly impacted the model's performance, each providing a different balance of performance.

In the custom algorithm, which yielded the highest results, a balance was achieved between accuracy and processing time, with the most appropriate parameters and optimization methods selected. The model's hidden layers consisted of 128 and 64 neurons, respectively, using the ReLU activation function, and a dropout rate of 30% was applied after each layer to prevent overfitting. The Adam optimization algorithm, which provides dynamic learning rate adjustments and generally delivers fast and stable results, was chosen, with a learning rate of 0.001. The model was trained using the categorical crossentropy loss function suitable for multi-class classification problems. During training, 10 epochs and a batch size of 32 were used, allowing the model to learn more efficiently by dividing the dataset into small batches for each epoch. In the feature extraction stage, 5000 features were extracted from the text data using the TF-IDF method; then, low-information features were eliminated using a variance threshold value of 0.0005. Additionally, dimensionality reduction was evaluated using PCA with 100 principal components. The class imbalance in the dataset was resolved using the SMOTE method, increasing the samples

of minority classes. The model's performance was tested using a 5-fold Stratified K-Fold cross-validation method and evaluated with metrics such as accuracy, precision, recall, F1-Score, AUC, and specificity. These parameters and optimization methods aimed to enhance the model's generalization capability and optimize processing time.

Future studies may test the model's performance with larger and more diverse datasets, different from those used in our study. Specifically, comparing the model's success on balanced, imbalanced, and highly imbalanced class structures can be beneficial to measure the effectiveness of methods developed for class imbalance. Furthermore, future work can evaluate PCA configurations that retain more information while still reducing dimensionality, aiming to improve the trade-off between efficiency and classification performance. Higher results for metrics such as accuracy, precision, recall, and F1-Score can be targeted this way. Additionally, different feature extraction and dimensionality reduction methods (e.g., UMAP or t-SNE) can also be tested in this process. Beyond this, using more advanced algorithms and models is recommended. For example, transformer-based models (e.g., BERT, RoBERTa) or hybrid models (e.g., BiLSTM-CNN) may achieve stronger results for text classification problems. Applying techniques such as grid search or Bayesian optimization for hyperparameter optimization may also enhance model performance. Future studies may focus on methods that increase model performance while reducing processing times. Such innovative approaches will significantly improve not only model performance but also its applicability and generalizability.

Conflict of Interest Statement

The authors declare no conflict of interest in this study.

References

[1] Çiğdem Acı and Adem Çırak. Türkçe haber metinlerinin konvolüsyonel sinir ağları ve word2vec kullanılarak sınıflandırılması. *Bilişim Teknolojileri Dergisi*, 12(3):219–228, 2019.

[2] Göksu Akgül, Ali Akın Çelik, Zeliha Ergül Aydın, and Zehra Kamışlı Öztürk. Hipotiroidi hastalığı teşhisinde sınıflandırma algoritmalarının kullanımı. *Bilişim Teknolojileri Dergisi*, 13(3):255–268, 2020.

[3] Güler Alparslan and Mahir Dursun. Konvolüsyonel sinir ağları tabanlı türkçe metin sınıflandırma. *Bilişim Teknolojileri Dergisi*, 16(1):21–31, 2023.

[4] Sinan Altun and Ahmet Alkan. Mr spektroskopi kullanılarak beyin tümörü tespitinde lstm tabanlı derin öğrenme uygulaması. *Gazi Üniversitesi Mühendislik Mimarlık Fakültesi Dergisi*, 38(2):1193–1202, 2022.

[5] Ali Arı, Furkan Ayaz, and Davut Hanbay. Emg sinyallerinin kısa zamanlı fourier dönüşüm özellikleri kullanılarak yapay sinir ağları ile sınıflandırılması. *Fırat Üniversitesi Mühendislik Bilimleri Dergisi*, 31(2):443–451, 2019.

[6] Saadet Aytaç Arpacı and Oya Kalıpsız. Yazilim hata siniflandirmasinda farkli naive bayes tekniklerin kiyaslanmasi. *Niğde Ömer Halisdemir Üniversitesi Mühendislik Bilimleri Dergisi*, 7(1):1–13, 2018.

[7] Serpil Aslan. Bigru-cnn tabanlı derin öğrenme modeliyle türkiye'deki covid-19 aşılarına yönelik twitter duygu analizi. *International Journal of Pure and Applied Sciences*, 8(2):312–330, 2022.

[8] Suat Atan and Yetkin Çınar. Borsa istanbul'da finansal haberler ile piyasa değeri ilişkisinin metin madenciliği ve duygu (sentiment) analizi ile incelenmesi. *Ankara Üniversitesi SBF Dergisi*, 74(1):1–34, 2019.

[9] Cahit Aydemir, Mehmet Kaya, et al. Küreselleşme kavrami ve ekonomik yönü. *Elektronik Sosyal Bilimler Dergisi*, 6(20):260–282, 2007.

[10] Emrah Aydemir, Murat Işık, and Türker Tuncer. Türkçe haber metinlerinin çok terimli naive bayes algoritması kullanılarak sınıflandırılması. *Fırat Üniversitesi Mühendislik Bilimleri Dergisi*, 33(2):519–526, 2021.

[11] Salih Aydın and Mehmet Serkan Çalı. Covid-19'un bireysel yatırımcıların yatırım davranışları üzerindeki etkisi: Covid-19 öncesi ve covid-19 sürecine yönelik karşılaştırmalı bir analiz. *Fiscaoeconomia*, 7(1):62–85, 2023.

[12] Sevgi Aytekin and Sema Dube. Piyasalar arasi dinamikler: Hisse senedi, tahvil, döviz ve emtia piyasalari arasindaki etkileşim ve nedensellik ilişkileri. *Elektronik Sosyal Bilimler Dergisi*, 15(59):1311–1326, 2016.

[13] Bilge Özlüer Başer, Metin Yangın, and E Selin Sarıdaş. Makine öğrenmesi teknikleriyle diyabet hastalığının sınıflandırılması. *Süleyman Demirel Üniversitesi Fen Bilimleri Enstitüsü Dergisi*, 25(1):112–120, 2021.

[14] Serkan Bulut. Dijital çağda medya: Makine öğrenmesi, algoritmik habercilik ve gazetecilikte işlevsiz insan sorunsali. *Selçuk İletişim*, 13(1):294–313, 2020.

[15] Serhat Burmaoğlu, Erkan Oktay, and Üstün Özen. Birleşmiş milletler kalkınma programı beşeri kalkınma endeksi verilerini kullanarak diskriminant analizi ve lojistik regresyon analizinin sınıflandırma performanslarının karşılaştırılması. *Savunma Bilimleri Dergisi*, 8(2):23–49, 2009.

[16] Birkan Büyükarıkan and Erkan Ülker. Aydinlatma özniteliği kullanilarak evrişimsel sinir aği modelleri ile meyve siniflandirma. *Uludağ Üniversitesi Mühendislik Fakültesi Dergisi*, 25(1):81–100, 2020.

[17] Berat Çağlar and Uğur Yavuz. Finansal haberlerin bitcoin fiyatlarına etkisinin yapay sinir ağları ile analizi. *Bilişim Teknolojileri Dergisi*, 14(1):65–78, 2021.

[18] Ünal Çağlar and Senem Dışkaya. Küreselleşme, uluslararasi para sistemi ve kriz. *Journal of Economic Policy Researches*, 5(2):1–24, 2018.

[19] Esra Çelik, Deniz Dal, and Tolga Aydin. Duygu analizi için veri madenciliği sınıflandırma algoritmalarının karşılaştırılması. *Avrupa Bilim ve Teknoloji Dergisi*, (27):880–889, 2021.

[20] Özer Çelik and Burak Can Koç. Tf-idf, word2vec ve fasttext vektör model yöntemleri ile türkçe haber metinlerinin sınıflandırılması. *Dokuz Eylül Üniversitesi Mühendislik Fakültesi Fen ve Mühendislik Dergisi*, 23(67):121–127, 2021.

[21] Orhan Cengiz. Küresel finans sisteminin ekonomi politiğinde renminbinin uluslararasılaşma süreci. *Gazi İktisat ve İşletme Dergisi*, 8(1):61–79, 2022.

[22] Özge Cömert and Mesut Toğaçar. Telepazarlama çağrılarının başarısını tahmin etmek üzere veriye dayalı bir yaklaşım. *Verimlilik Dergisi*, 57(4):735–746, 2023.

[23] Mehmet Fatih Çömlekçi. Sosyal medyada dezenformasyon ve haber doğrulama platformlarinin pratikleri. *Gümüşhane Üniversitesi İletişim Fakültesi Elektronik Dergisi*, 7(3):1549–1563, 2019.

[24] Mehmet Cem Danacı and Özlem Çetintaş. Bankalarda finansal teknoloji ve yenilikler. *Turkish Business Journal*,

1(2):179–187, 2020.

[25] Ali Dayan and Atınç Yılmaz. Doğal dil işleme ve derin öğrenme algoritmaları ile makine dili modellemesi. *Dicle Üniversitesi Mühendislik Fakültesi Mühendislik Dergisi*, 13(3):467–475.

[26] Hasan Demir, Pakize Erdoğmuş, and Meral Kekeçoğlu. Destek vektör makineleri, ysa, k-means ve knn kullanarak arı türlerinin sınıflandırılması. *Düzce Üniversitesi Bilim ve Teknoloji Dergisi*, 6(1):47–67, 2018.

[27] Mustafa Demirbilek and Sevim Özulukale Demirbilek. Google yorumları üzerinden makine öğrenme yöntemleri ve amazon comprehend ile duygu analizi: İç anadoluda bir üniversite örneği. *Üniversite Araştırmaları Dergisi*, 6(4):452–461, 2023.

[28] Serkan Demirel. Küresel ekonomik, politik ve jeopolitik risklerin bireysel ve kurumsal finansal işlemlere etkisi. *Elektronik Sosyal Bilimler Dergisi*, 23(90):645–667, 2024.

[29] Gizem Dilki and Özlem Deniz Başar. İşletmelerin iflas tahmininde k-en yakin komşu algoritmasi üzerinden uzaklik ölçütlerinin karşilaştirilmasi. *İstanbul Ticaret Üniversitesi Fen Bilimleri Dergisi*, 19(38):224–233, 2020.

[30] Yeşim Dokuz, Aslı Bozdağ, and Begüm Gökçek. Hava kalitesi parametrelerinin tahmini ve mekansal dağilimi için makine öğrenmesi yöntemlerinin kullanilmasi. *Niğde ömer halisdemir üniversitesi mühendislik bilimleri dergisi*, 9(1):37–47, 2020.

[31] Sabah Ekonomi. 3 haftada 1 milyon yatırımcı geldi, 2024. Accessed: November 27, 2024.

[32] Mehmet Bilal Er and İbrahim Işık. Lstm tabanlı derin ağlar kullanılarak diyabet hastalığı tahmini. *Türk Doğa ve Fen Dergisi*, 10(1):68–74, 2021.

[33] Gülnur Begüm Ergün, Selda Güney, and Tahsin Gürkan Ergün. Köpeklerdeki uzun kemiklerin evrişimsel sinir ağları kullanılarak sınıflandırılması. *Fırat Üniversitesi Mühendislik Bilimleri Dergisi*, 33(1):125–132, 2021.

[34] Junus Ganiev. Küreselleşme, finansal piyasalar ve kriz. *Journal of Economic Policy Researches*, 1(2), 2014.

[35] Serim Gençaslan, Anıl Utku, and M Ali Akcayol. Derin öğrenme tabanlı video üzerinde olay sınıflandırma. *Politeknik Dergisi*, 26(3):1155–1165, 2022.

[36] Yasin Görmez, Halil Arslan, and Bilal Atak. Türkçe metinlerde duygu analizi: Derin öğrenme yaklaşımlarının ve ön işlem süreçlerinin model performansına etkisi. *Fırat Üniversitesi Mühendislik Bilimleri Dergisi*, 36(1):509–520, 2024.

[37] Numan Halit Güldemir and Ahmet Alkan. Derin öğrenme ile optik koherens tomografi görüntülerinin sınıflandırılması. *Fırat Üniversitesi Mühendislik Bilimleri Dergisi*, 33(2):607–615, 2021.

[38] Cengiz Hark. Sahte haber tespiti için derin bağlamsal kelime gömülmeleri ve sinirsel ağların performans değerlendirmesi. *Fırat Üniversitesi Mühendislik Bilimleri Dergisi*, 34(2):733–742, 2022.

[39] Hulya Hark, Meral Karakurt, Cengiz Hark, and Ali Karci. Glove kelime gömmeleri ve sinir ağları ile haber metinlerinin sınıflandırılması. *International Journal of Pure and Applied Sciences*, 9(1):175–187, 2023.

[40] Ali İnan and Esra Var. Sınıflandırma için diferansiyel mahremiyete dayalı öznitelik seçimi. *Gazi Üniversitesi Mühendislik Mimarlık Fakültesi Dergisi*, 33(1), 2018.

[41] Murat İnce. Üstverilerin derin öğrenme algoritmaları kullanılarak otomatik olarak çıkartılması ve sınıflanması. *Düzce Üniversitesi Bilim ve Teknoloji Dergisi*, 9(6):177–187, 2021.

[42] Kaggle. Learn ai bbc dataset, 2024. https://www.kaggle.com/competitions/learn-ai-bbc/data (Accessed: November 27, 2024).

[43] Tahir Emre Kalaycı. Kimlik hırsızı web sitelerinin sınıflandırılması için makine öğrenmesi yöntemlerinin karşılaştırılması. *Pamukkale Üniversitesi Mühendislik Bilimleri Dergisi*, 24(5):870–878, 2018.

[44] Meral Karakurt, ALİ KARCI, et al. Glove kelime gömmeleri ve sinir ağları ile haber metinlerinin sınıflandırılması. *International Journal of Pure and Applied Sciences*, 9(1), 2023.

[45] Mehmet Kayakuş and Fatma Yiğit Açıkgöz. Twitter'da makine öğrenmesi yöntemleriyle sahte haber tespiti. *Abant Sosyal Bilimler Dergisi*, 23(2):1017–1027, 2023.

[46] Ali Cihat Kelle and Hüseyin Yüce. Mqtt trafiğinde dos saldırılarının makine öğrenmesi ile sınıflandırılması ve modelin shap ile yorumlanması. *Journal of Materials and Mechatronics: A*, 3(1):50–62, 2022.

[47] Zeynep Hilal Kilimci. Borsa tahmini için derin topluluk modellleri (dtm) ile finansal duygu analizi. *Gazi Üniversitesi Mühendislik Mimarlık Fakültesi Dergisi*, 35(2):635–650, 2020.

[48] Doğancan Kınık and Aysun Güran. Tf-idf ve doc2vec tabanlı türkçe metin sınıflandırma sisteminin başarım değerinin ardışık kelime grubu tespiti ile arttırılması. *Avrupa Bilim ve Teknoloji Dergisi*, (21):323–332, 2021.

[49] Sedat Metlek and Kiyas Kayaalp. Derin öğrenme ve destek vektör makineleri ile görüntüden cinsiyet tahmini. *Düzce Üniversitesi Bilim ve Teknoloji Dergisi*, 8(3):2208–2228, 2020.

[50] Gözde Nergiz and Erdinç Avaroğlu. Türkçe sosyal medya yorumlarındaki siber zorbalığın derin öğrenme ile tespiti. *Avrupa Bilim ve Teknoloji Dergisi*, (31):77–84, 2021.

[51] European Metrology Network. Jrp neurobiostand project, 2024. Accessed: November 20, 2024.

[52] Halil İbrahim Okur, Kadir Tohma, and Ahmet Sertbaş. Graf sinir ağları ile ilişkisel türkçe metin sınıflandırma. *Politeknik Dergisi*, pages 1–1.

[53] Feyza ALTUNBEY Özbay and Bilal Alataş. Çevrimiçi sosyal medyada sahte haber tespiti. *Dicle Üniversitesi Mühendislik Fakültesi Mühendislik Dergisi*, 11(1):91–103, 2020.

[54] Yusuf Bahri Özçelik and Aytaç Altan. Diyabetik retinopati teşhisi için fundus görüntülerinin derin öğrenme tabanlı sınıflandırılması. *Avrupa Bilim ve Teknoloji Dergisi*, (29):156–167, 2021.

[55] Ersin Özince. Finansal sektör–uluslararası gelişmeler ve türkiye deneyimi. *Muhasebe ve Finansman Dergisi*, (40):24–44, 2008.

[56] Mustafa Özkan and Görkem Kar. Türkçe dilinde yazilan bilimsel metinlerin derin öğrenme tekniği uygulanarak çoklu siniflandirilmasi. *Mühendislik Bilimleri ve Tasarım Dergisi*, 10(2):504–519, 2022.

[57] Özlem Özmen, Ahmad Khdr, and Engin Avcı. Sınıflandırıcıların kalp hastalığı verileri üzerine performans karşılaştırması. *Fırat Üniversitesi Mühendislik Bilimleri Dergisi*, 30(3):153–159, 2018.

[58] Ali Öztürk, Üsame Durak, and Fatma Badıllı. Twitter verilerinden doğal dil işleme ve makine öğrenmesi ile hastalik tespiti. *Konya Journal of Engineering Sciences*, 8(4):839–852, 2020.

[59] Ruşen Ramizoğlu. Modern makine çevirisi teknolojilerine

yönelik bir karşilaştirma. *Uluslararası Türkçe Edebiyat Kültür Eğitim (TEKE) Dergisi*, 13(1):126–137, 2024.

[60] Ensar Arif Sağbaş and Serkan Ballı. Akıllı saat algılayıcıları ile insan hareketlerinin sınıflandırılması. *Süleyman Demirel Üniversitesi Fen Bilimleri Enstitüsü Dergisi*, 21(3):980–990, 2017.

[61] Durmuş Özkan ŞAHİN, Nurullah ATEŞ, and Erdal KILIÇ. Metin sınıflandırmada öznitelik seçimi feature selection in text classification.

[62] Önder Şahinaslan, Hüseyin Dalyan, and Ender Şahinaslan. Naive bayes sınıflandırıcısı kullanılarak youtube verileri üzerinden çok dilli duygu analizi. *Bilişim Teknolojileri Dergisi*, 15(2):221–229, 2022.

[63] Mehmet Umut Salur and İlhan Aydın. Türkçe tweetler için derin özellik çıkarımı tabanlı yeni bir duygu sınıflandırma modeli. *Fırat Üniversitesi Mühendislik Bilimleri Dergisi*, 34(1):1–13, 2022.

[64] Yunus Santur. Derin öğrenme ve aşağı örnekleme yaklaşımları kullanılarak duygu sınıflandırma performansının iyileştirilmesi. *Fırat Üniversitesi Mühendislik Bilimleri Dergisi*, 32(2):561–570, 2020.

[65] Mehmet Saraç and Mehmet Burak Kahyaoğlu. Bireysel yatırımcıların risk alma eğilimine etki eden sosyo-ekonomik ve demografik faktörlerin analizi. *BDDK Bankacılık ve Finansal Piyasalar Dergisi*, 5(2):135–157, 2011.

[66] Ali Tezcan Sarızeybek and Onur Sevli. Makine öğrenmesi yöntemleri ile banka müşterilerinin kredi alma eğiliminin karşılaştırmalı analizi. *Journal of Intelligent Systems: Theory and Applications*, 5(2):137–144, 2022.

[67] Gözde Şişmanoğlu, Furkan Koçer, Mehmet Ali Önde, and Ozgur Koray Sahingoz. Derin öğrenme yöntemleri ile borsada fiyat tahmini. *Bitlis Eren Üniversitesi Fen Bilimleri Dergisi*, 9(1):434–445, 2020.

[68] Ebru Somuncu and Nesrin Aydın Atasoy. Evrişimli tekrarlayan sinir ağı ile metin görüntüleri üzerinde karakter tanıma uygulaması gerçekleştirilmesi. *Gazi Üniversitesi Mühendislik Mimarlık Fakültesi Dergisi*, 37(1):17–28, 2022.

[69] Süleyman Gökhan Taşkın, Ecir Uğur Küçüksille, and Kamil Topal. Twitter üzerinde türkçe sahte haber tespiti. *Balıkesir Üniversitesi Fen Bilimleri Enstitüsü Dergisi*, 23(1):151–172, 2021.

[70] Abdülkadir TEPECİK and Engin DEMİR. Türkçe ses kayıt verilerinin countvectorizer ve tf-idfvectorizer yöntemleri ile bert modelleri olarak google colab platformunda ve rapidminer'da makine öğrenmesi algoritmalarıyla analizi. *Firat University Journal of Science*, 34(1), 2022.

[71] Mesut Toğaçar, Kamil Abdullah Eşidir, and Burhan Ergen. Yapay zekâ tabanlı doğal dil işleme yaklaşımını kullanarak internet ortamında yayınlanmış sahte haberlerin tespiti. *Journal of Intelligent Systems: Theory and Applications*, 5(1):1–8, 2021.

[72] Enver Yazğılı and Muhammet Baykara. Türkçe metinlerde makine öğrenmesi yöntemleri ile siber zorbalık tespiti. *Gümüşhane Üniversitesi Fen Bilimleri Dergisi*, 12(2):443–453, 2022.

[73] Mehmet Can Yılmaz and Zeynep Orman. Lstm derin öğrenme yaklaşımı ile covid-19 pandemi sürecinde twitter verilerinden duygu analizi. *Acta Infologica*, 5(2):359–372, 2021.

[74] Nurcan Yücel and Özge Cömert. Müşteri duyarlılığını keşfetmek için yapay zeka destekli analiz ile çevrimiçi ürün incelemelerinden anlamlı bilgiler elde etme. *Fırat Üniversitesi Mühendislik Bilimleri Dergisi*, 35(2):679–690, 2023.

[75] Veli Özgür Yurtdaşseven and Serkan Çankaya. Finansal tavsiyelerin yatırımcı davranışlarına etkisi. *Finans Ekonomi ve Sosyal Araştırmalar Dergisi*, 3(2):564–575, 2018.

[76] Adham Jolosı Jolosı Zada and Ahmet Albayrak. Duygu analizi ve topluluk öğrenmesi yaklaşımları ile kullanıcı yorumlarının analizi. *Düzce Üniversitesi Bilim ve Teknoloji Dergisi*, 11(4):1725–1732, 2023.

[77] Ankur Zing et al. Aspect-based sentiment analysis for financial news, 2024. Accessed: 2024-11-30.